PROJECT DOCUMENTATION

Project Name:

Secure CI/CD Pipeline with Zero-Downtime Deployment on AWS

Developed by:

Mukhesh DN

Muppidi Sai Adhitya

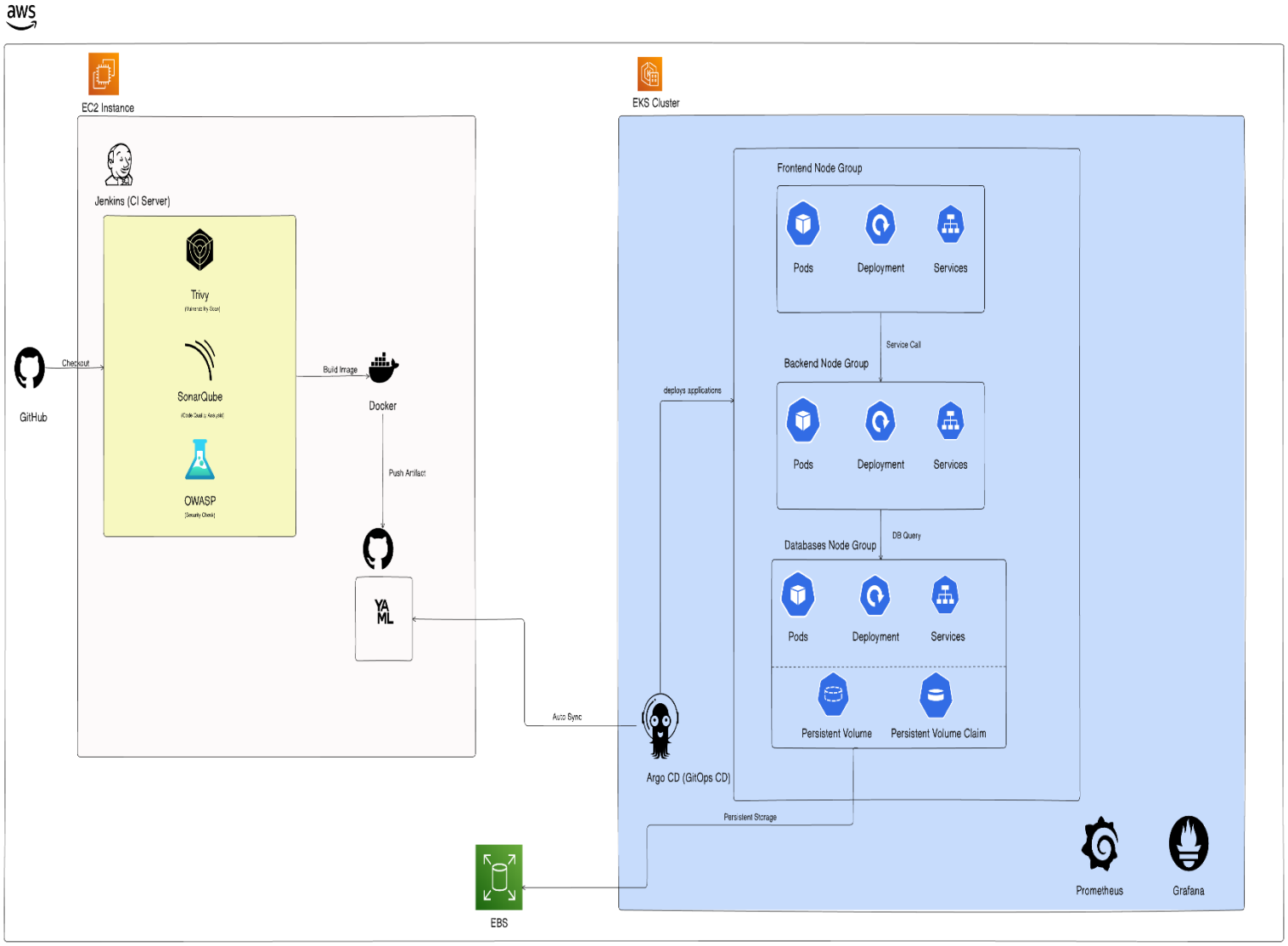
Purpose Of This Documentation

The purpose of this document is to serve as a detailed reference for the design, implementation, and execution of the DevSecOps pipeline. It captures:

- The high-level architecture and workflow of the CI/CD process
- The tools and technologies used at each stage
- Step-by-step procedures followed during implementation
- Commands executed and configurations applied
- Security controls and quality checks integrated into the pipeline
- Deployment strategy and GitOps workflow

This documentation is **intended** for developers, DevOps engineers, security engineers, and stakeholders who want to understand, replicate, maintain, or enhance the implemented DevSecOps solution. It also serves as a knowledge base for onboarding new team members and for future audits or enhancements of the pipeline.

High level design



Docker Setup

PreRequisites

Before proceeding with the Docker setup and containerization of the application, ensure that the following prerequisites are in place:

1. **Docker Hub Account**
   An active Docker Hub account is required to store and manage container images for both frontend and backend services.
2. **Separate Docker Hub Repositories (Frontend & Backend)**
   Dedicated Docker Hub repositories for the frontend and backend services to enable independent image versioning and deployment.
3. **Docker Desktop**
   Docker Desktop must be installed and running on the local development machine or build server for building and testing Docker images.
4. **Docker CLI**
   Docker Command Line Interface (CLI) installed and accessible from the terminal to execute build, tag, login, and push commands.

Clone the Source Code from GitHub

Link: https://github.com/MukheshDN4352/DevSecOps-Project.git

Dockerfile Design and Configuration Overview

1. We used an open-source **three-tier application** named **Social Echo** as the base application for this project.

2. A **multi-stage Docker build** approach was implemented to reduce the final **Docker image size** and improve build efficiency.

3. For the frontend service, **Nginx** was used as the **web server**.
   a. The lightweight **Alpine-based Nginx image (nginx:alpine)** was selected as the runtime base image to minimize resource usage.
   b. **Nginx** was configured to serve the **frontend application**.

4. All required **Dockerfiles** for the frontend and backend services, along with the **Nginx configuration file (nginx.conf)**, are provided below for reference.

FrontEnd Dockerfile

```
FROM node:18-alpine AS build
# Set working directory
WORKDIR /app
# Copy dependency files
COPY package*.json ./
# Install dependencies (clean, reproducible install)
RUN npm ci
# Add Babel plugin (for CRA compatibility with private props)
RUN npm install --save-dev @babel/plugin-proposal-private-property-in-object
# Copy source code
COPY . .
# Build production assets
RUN npm run build


#-------------------------------------------------
FROM nginx:alpine
# Copy built static files from build stage
COPY --from=build /app/build /usr/share/nginx/html
# Remove default nginx config and replace with a custom one
COPY nginx.conf /etc/nginx/conf.d/default.conf
# Expose port 80
EXPOSE 80
# Start Nginx in the foreground
CMD ["nginx", "-g", "daemon off;"]
```

Nginx config file

```
server {
  listen 80;
  server_name _;

  root /usr/share/nginx/html;
  index index.html;

  # Handle SPA routes (React Router)
  location / {
    try_files $uri /index.html;
  }

  # Proxy API calls to backend
  location /api/ {
    proxy_pass http://backend:4000;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection 'upgrade';
    proxy_set_header Host $host;
    proxy_cache_bypass $http_upgrade;
  }

  # Error handling
  error_page 500 502 503 504 /50x.html;
  location = /50x.html {
    root /usr/share/nginx/html;
  }
}
```

Backend Dockerfile

```
# ---- Stage 1: Build dependencies ----
FROM node:18-alpine AS build
# Set working directory
WORKDIR /usr/src/app
# Copy dependency files first (for better caching)
COPY package*.json ./
# Install all dependencies (including dev)
RUN npm install
# Copy the application code
COPY . .


# ---- Stage 2: Production runtime ----
FROM node:18-alpine
# Set working directory
WORKDIR /usr/src/app
# Copy only production dependencies from the build stage
COPY --from=build /usr/src/app/node_modules ./node_modules
# Copy source code from build stage
COPY --from=build /usr/src/app ./
# Expose port (backend runs on 4000)
EXPOSE 4000
# Set environment variable for production
ENV NODE_ENV=production
# Start the app using npm production script
CMD ["npm", "run", "production"]
```

Jenkins Setup

PreRequisites

- **AWS Account**
  An active AWS account to provision and manage the Jenkins server (EC2 instance) and related resources.

- **EC2 Instance**
  An Amazon EC2 instance (Linux-based) to host the Jenkins server.

- **IAM Role or User Credentials**
  AWS IAM role or user with required permissions to:
  Access EC2

Jenkins Setup and Pipeline Overview

In this project, **Jenkins** is hosted on an **AWS EC2 instance** and accessed through its **public IP address**. Jenkins is used as the core **CI engine** to automate the build, security scanning, and container image creation processes.

A **canary deployment model** is implemented to enable safe and incremental releases. New application versions are first deployed to a **canary environment** for validation before being promoted to the **stable environment**.

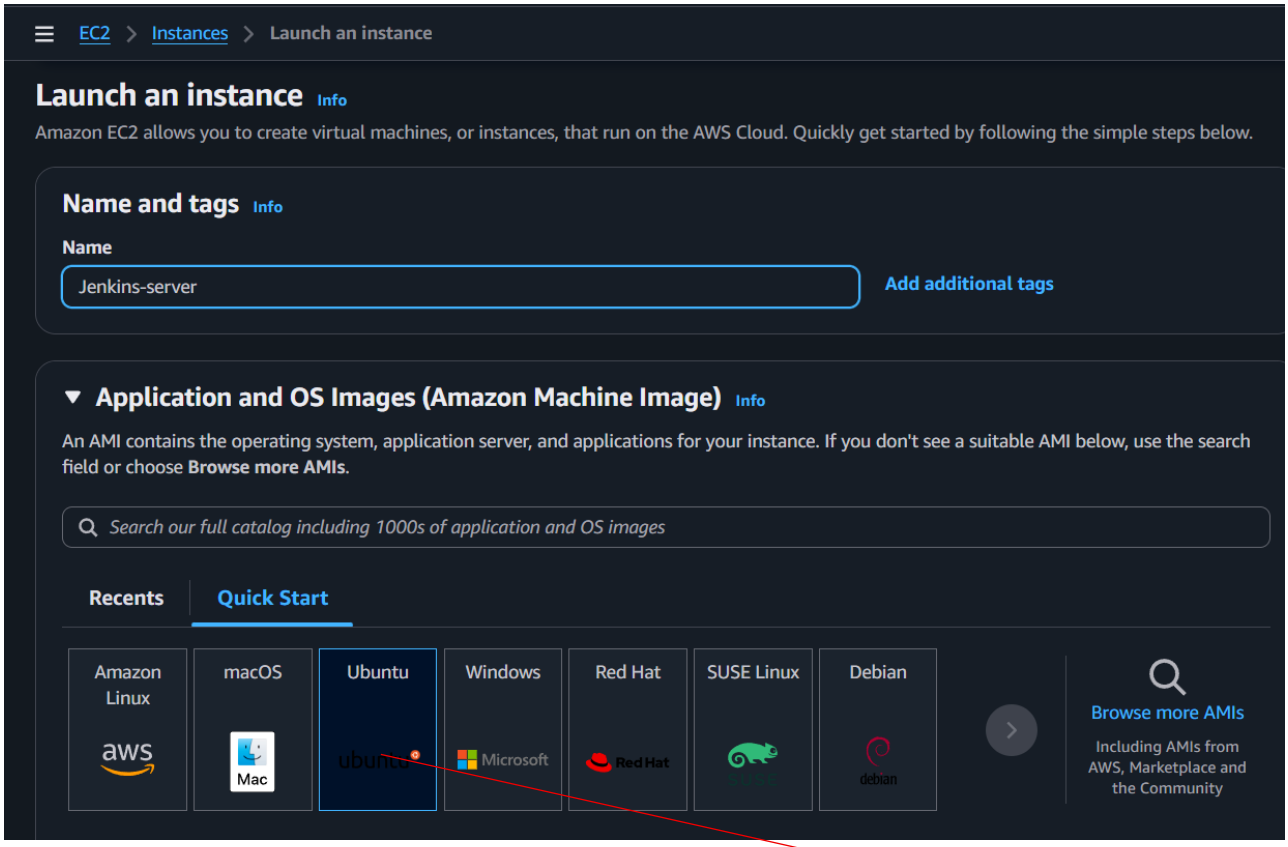
The Jenkins pipeline consists of the following stages:

- **Clone Code from GitHub**
- **Promote Existing Canary → Stable**
- **SonarQube Quality Analysis**
- **OWASP Dependency Check**
- **Sonar Quality Gate**
- **Trivy File System Scan**
- **Build & Push Docker Images**
- **Update Canary with New Images**
- **Commit & Push to GitHub (for ArgoCD Sync)**

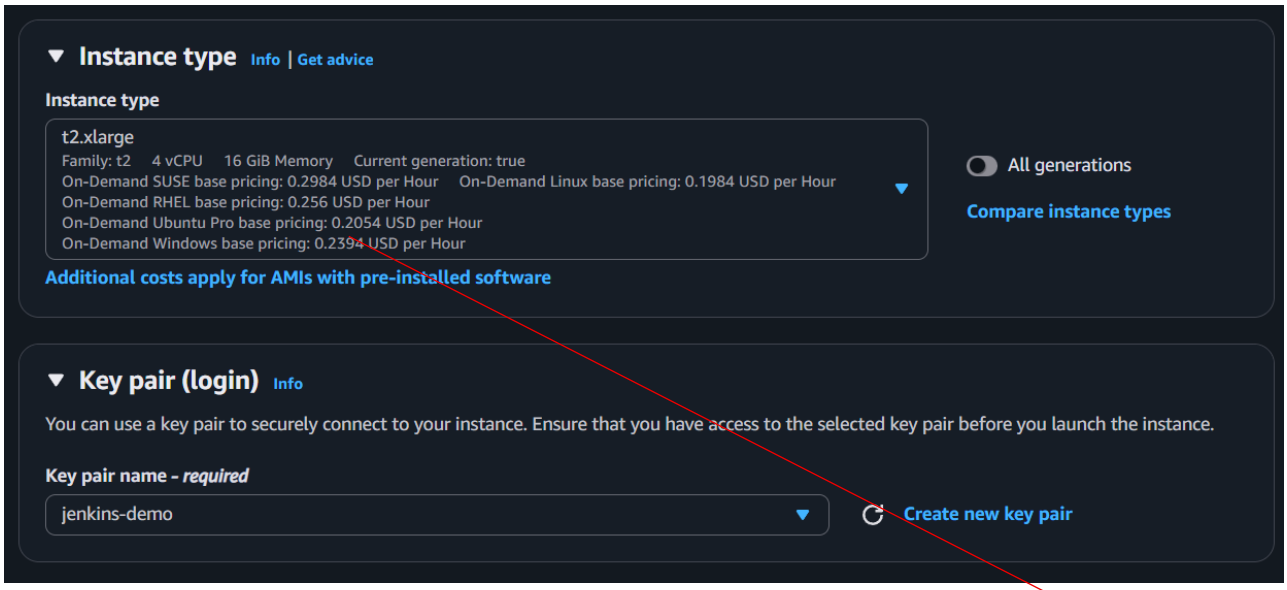
Post-build actions include **email notifications** for both **successful** and **failed** pipeline executions.

All required **Jenkins credential configurations**, **plugins installation**, **pipeline configurations**, **EC2 instance setup commands**, and **GitHub webhook configuration for automatic build triggers** are provided in the sections below.

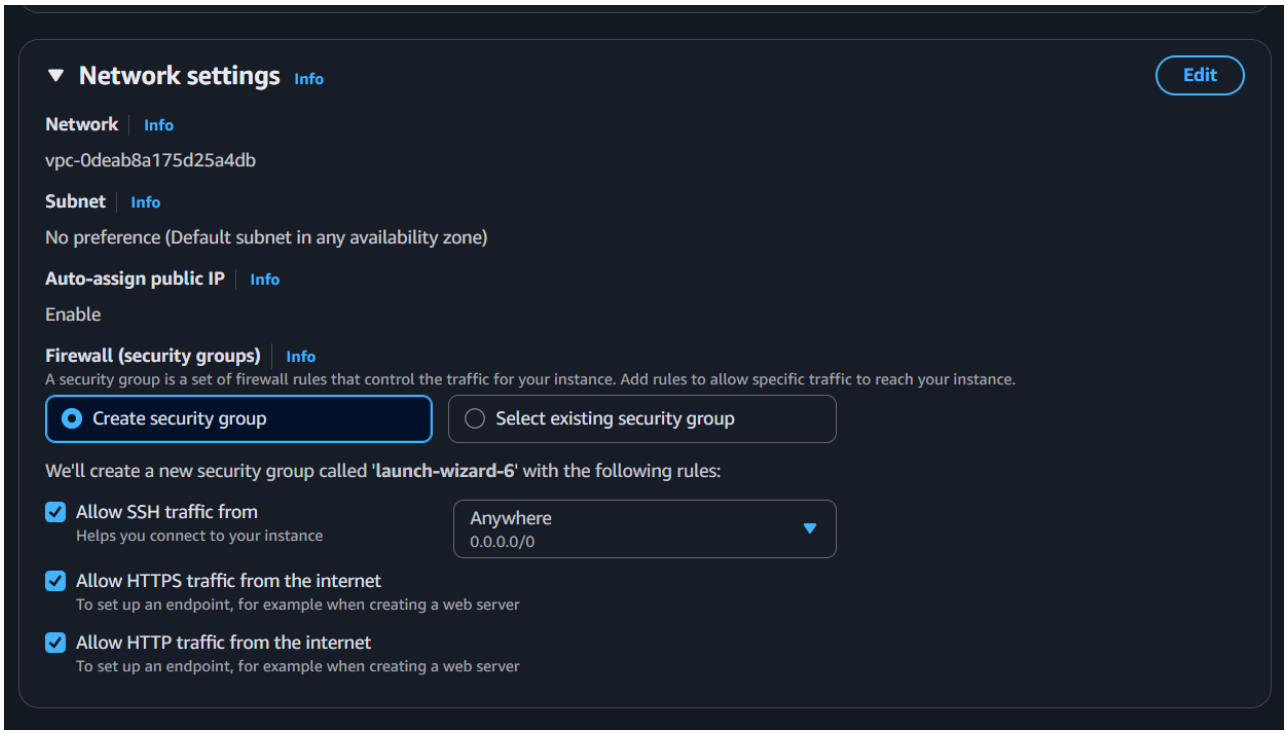
Creating EC2 instance in AWS

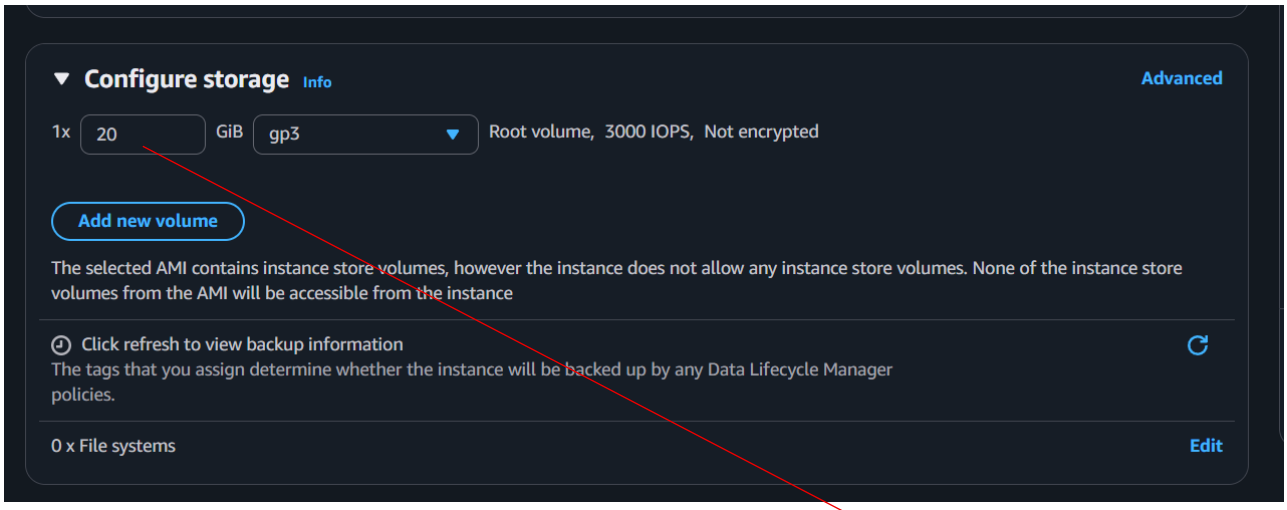


Select **ubuntu** AMI

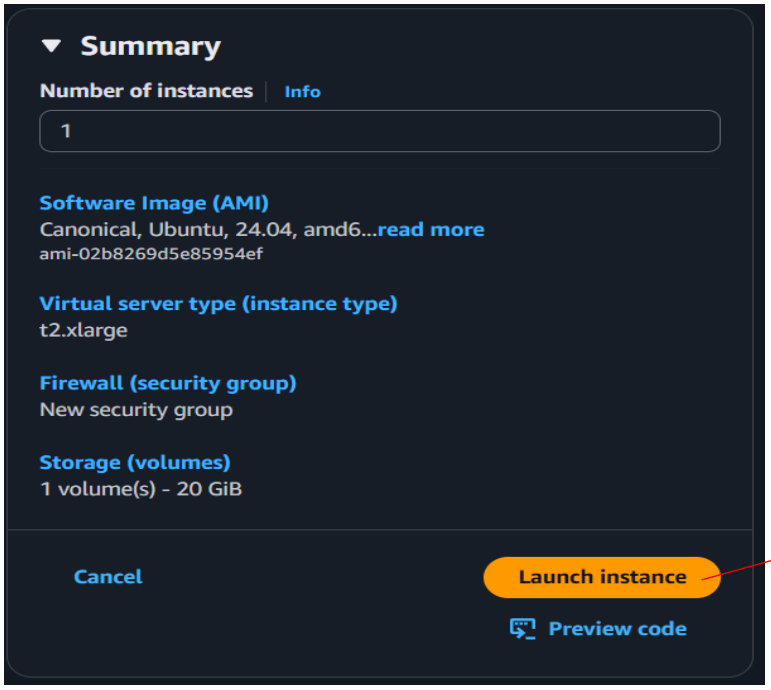


Select **T2.xlarge** instance type

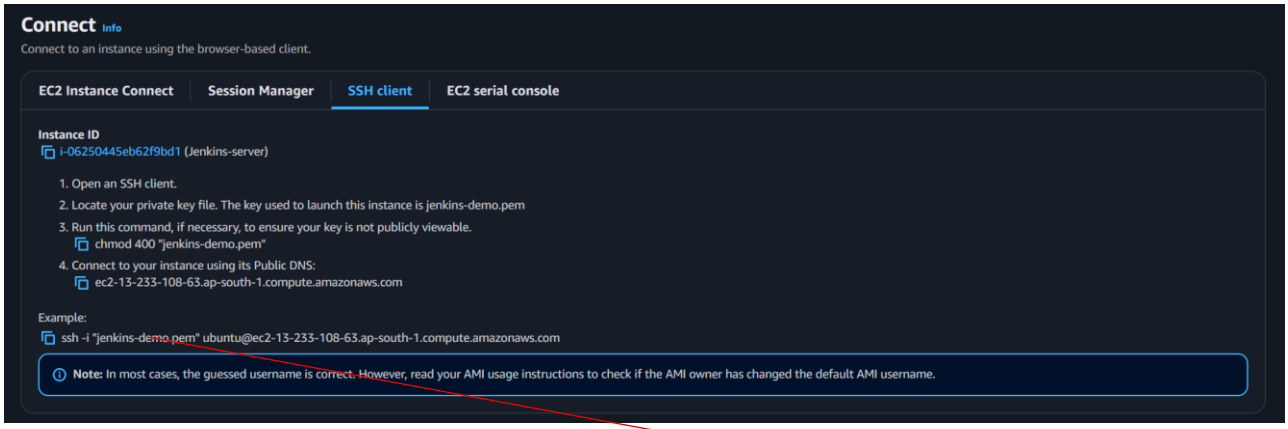




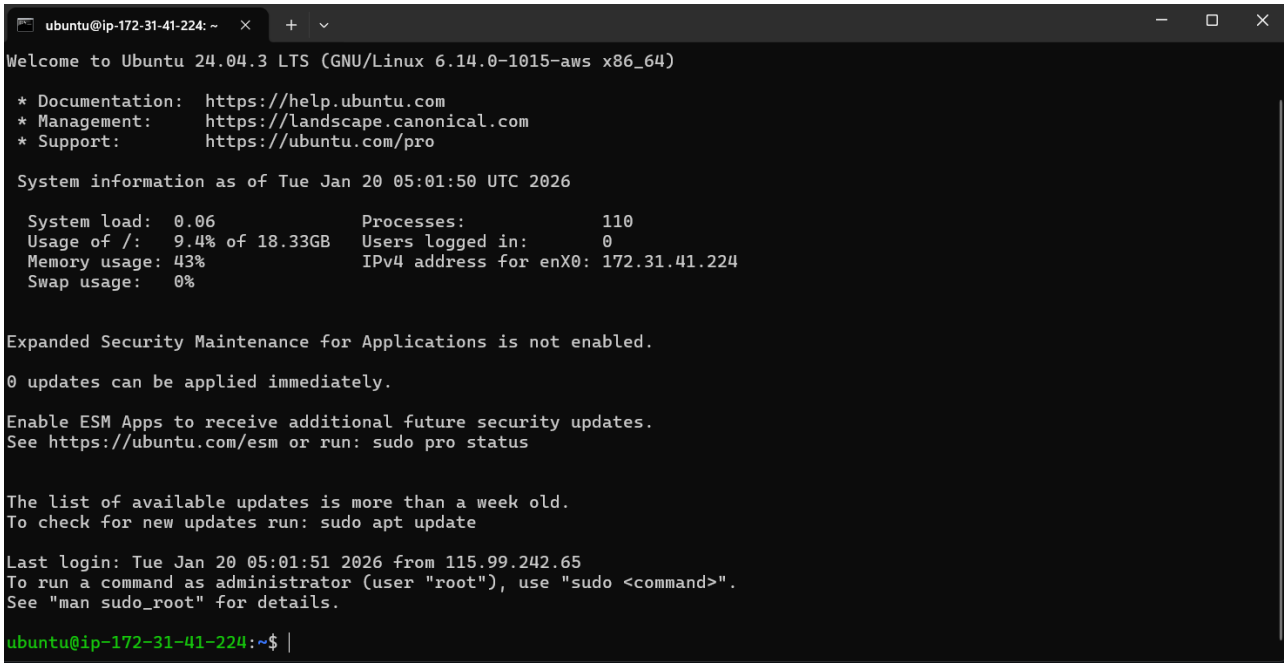
Take **20 GIB** volume



Click on launch instance



Copy this command to connect to the instance from your cmd



The next set of commands needs to be executed on the Ubuntu EC2 instance.

Jenkins-server Configuration Commands

sudo apt-get update

 # docker installation

1. sudo apt-get install docker.io -y
2. sudo apt-get install docker-compose -y
3. sudo usermod -aG docker ubuntu
4. sudo reboot
5. docker ps

 # jenkins installation

1. sudo apt update
2. sudo apt install fontconfig openjdk-21-jre
3. java -version

4. sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc
   https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
   echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]" \
   https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
   /etc/apt/sources.list.d/jenkins.list > /dev/null

5. sudo apt update

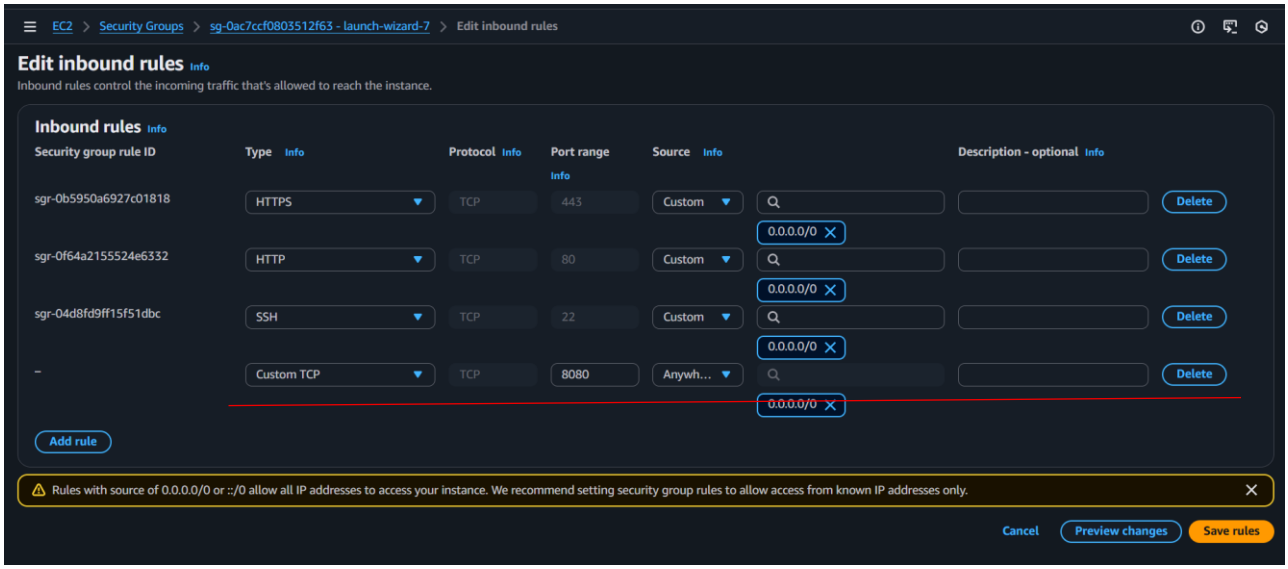
6. sudo apt install jenkins

7. sudo systemctl status jenkins

8. wq

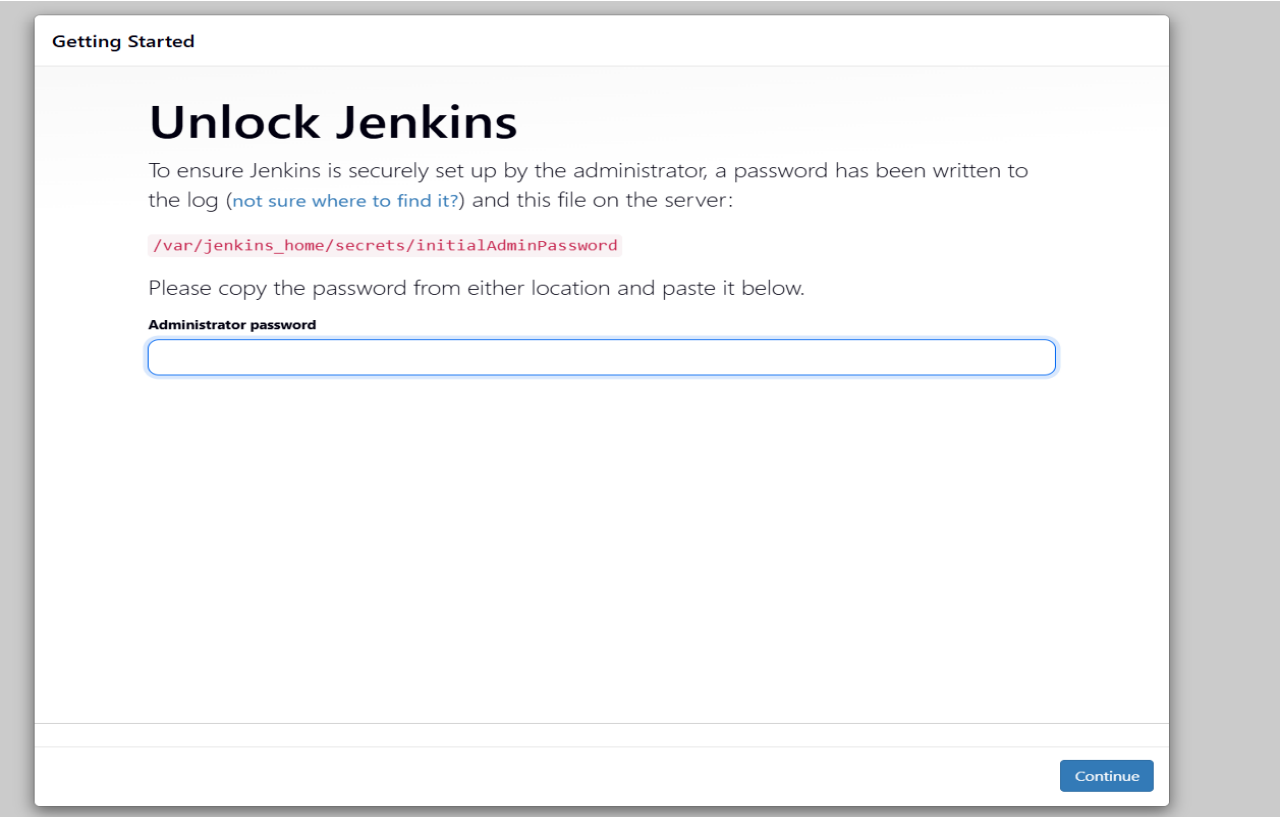
Allow inbound traffic to the instance

aws → ec2 instance →security groups →edit inbound rules → allow port 8080



Access the jenkins application through

http://<publlic IP of the instance>:8080



sudo cat /var/lib/jenkins/secrets/initialAdminPassword

pulgins to install

sonarQube Scanner

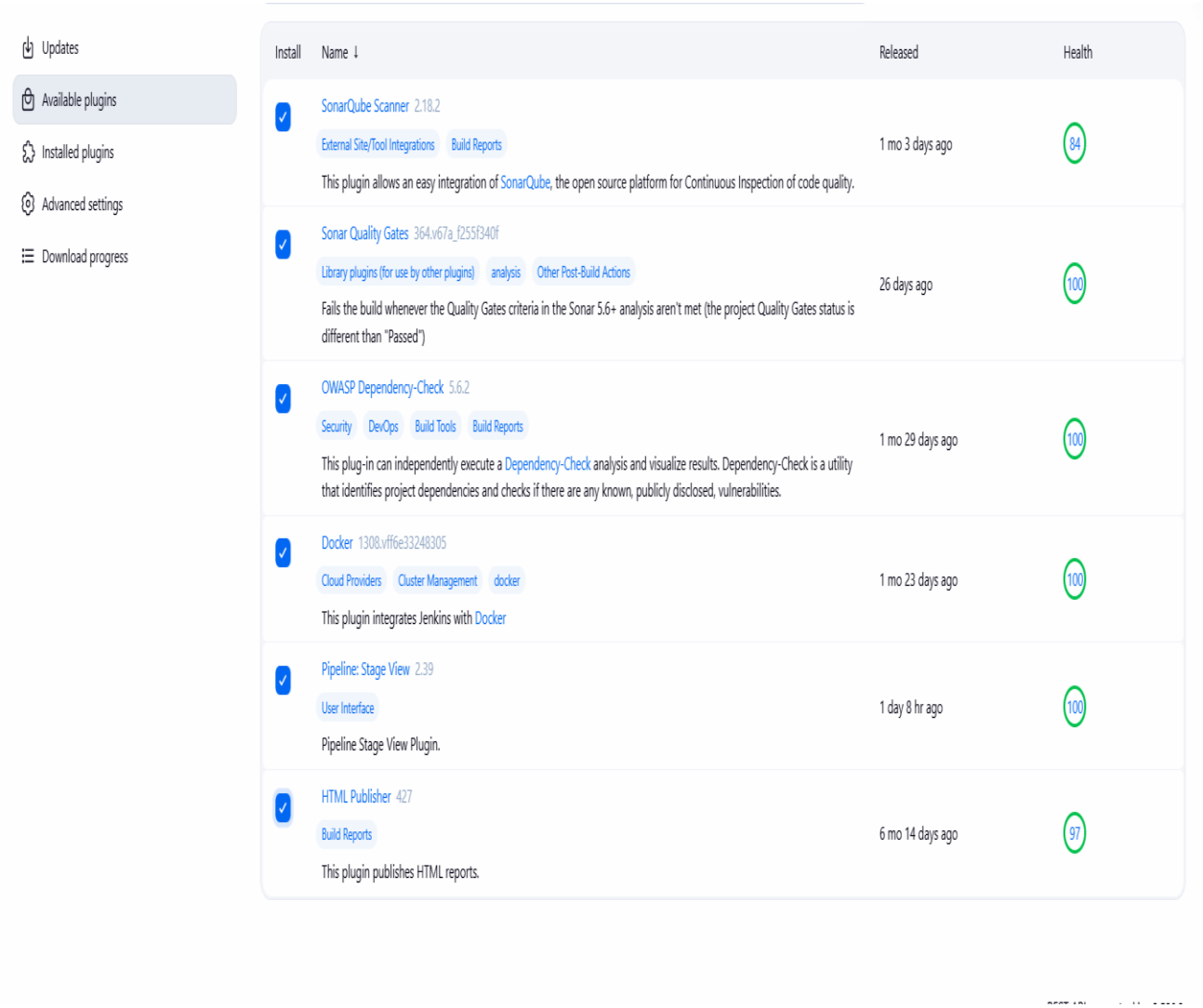
SonarQuality gates

owasp dependency check

docker

stage view

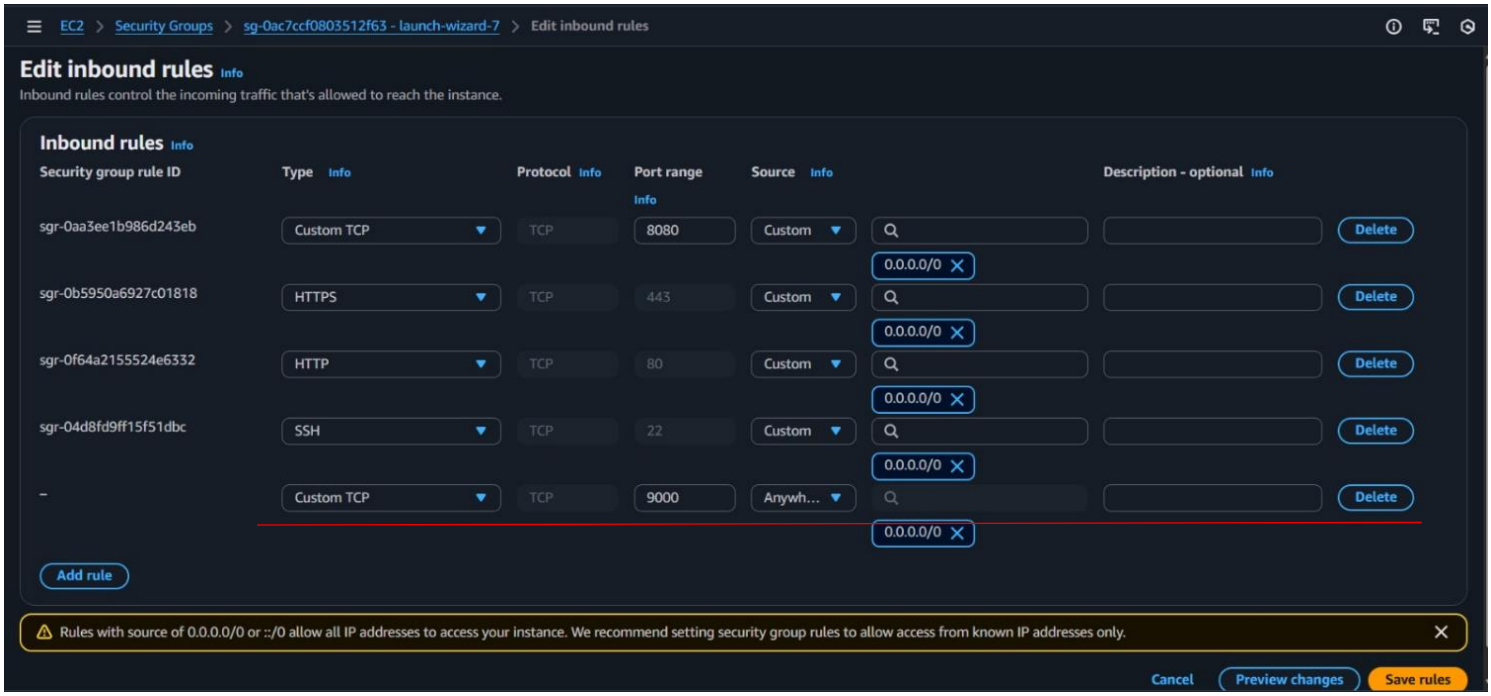
HTML publisher (for Trivy)



Sonarqube Setup

docker run -itd --name sonarqube-server -p 9000:9000 sonarqube:lts-community

aws ---> ec2 instance --> security groups --->edit inbound rules ---> allow port 9000



Trivy installation

- sudo apt-get install wget apt-transport-https gnupg lsb-release

- wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key | \
  gpg --dearmor | sudo tee /usr/share/keyrings/trivy.gpg

- echo "deb [signed-by=/usr/share/keyrings/trivy.gpg] https://aquasecurity.github.io/trivy-repo/deb \
  $(lsb_release -sc) main" | sudo tee /etc/apt/sources.list.d/trivy.list

- sudo apt-get update

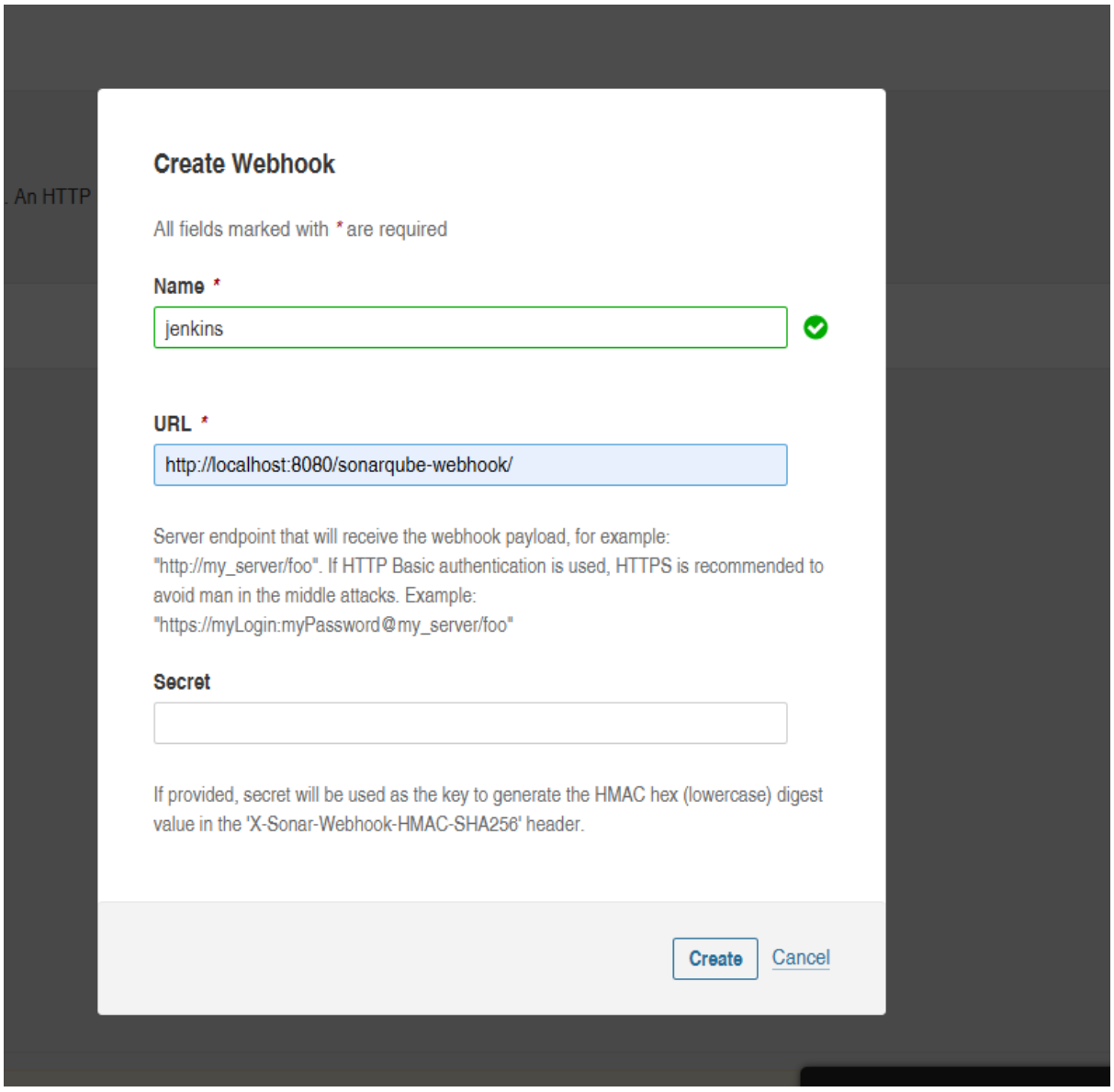
- sudo apt-get install trivy

Sonarqube and jenkins integeration

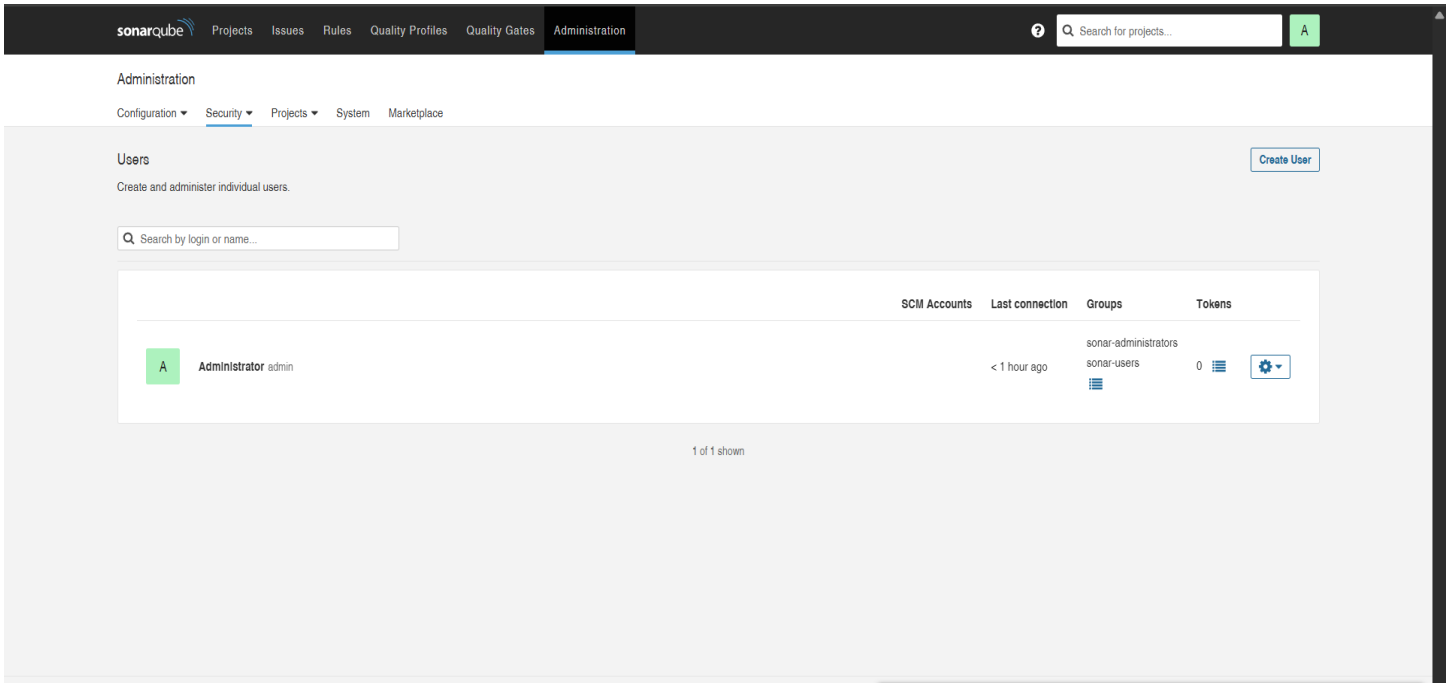
1)sonarqube → administrator →configuration → webhook → create →

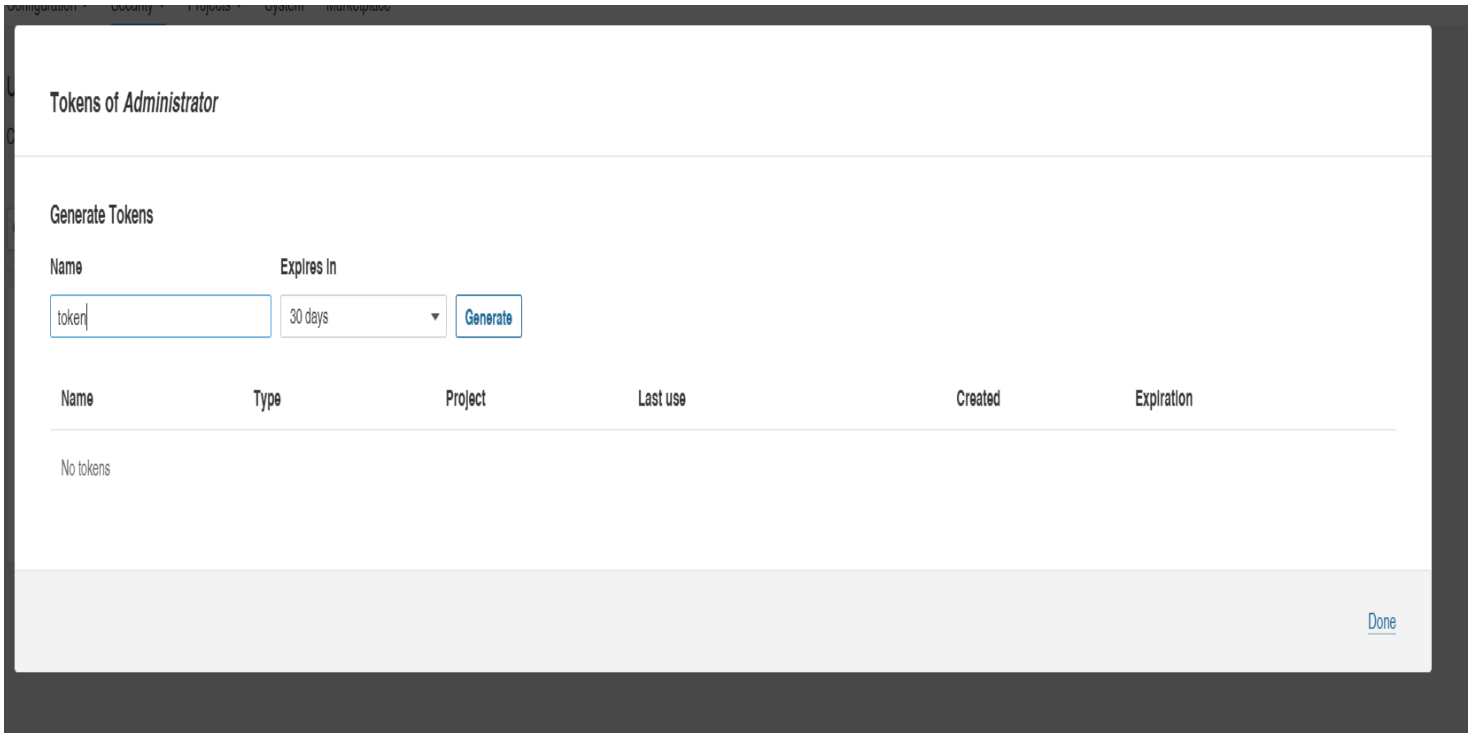
name: jenkins

url :http://35.154.237.213:8080/sonarqube-webhook/



2)sonarqube --> administrator --> security --> user --> generate token
[ex:squ_1d7e9d17fa42aebfaaab8846aefcbbedb8c33b60]

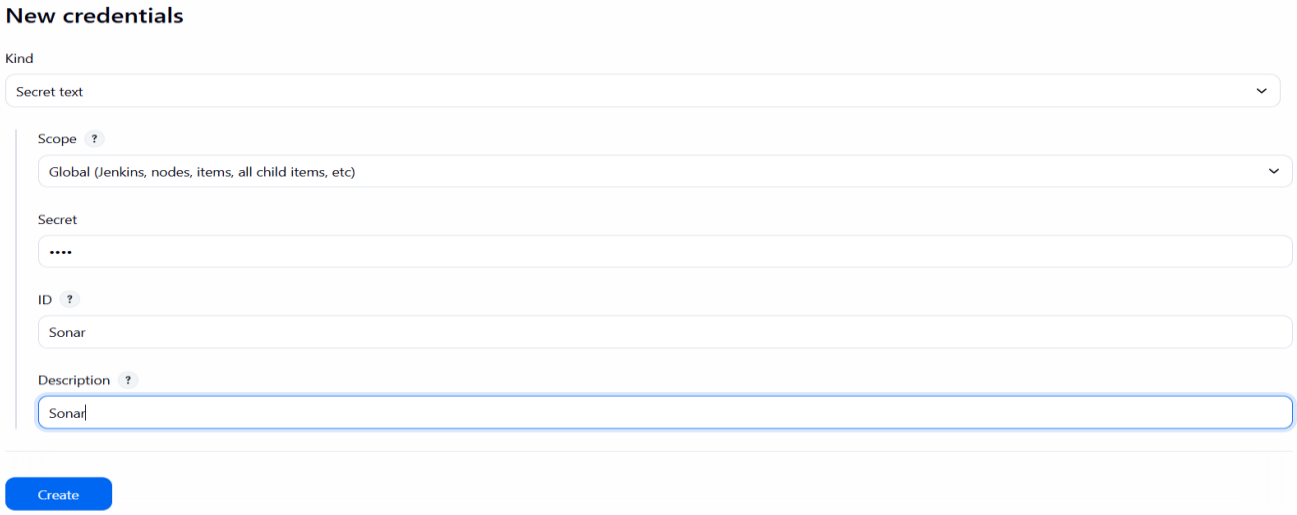




3)jenkins → credentials →global →add credentials →

[secret text] id : generated token,

description :Sonar

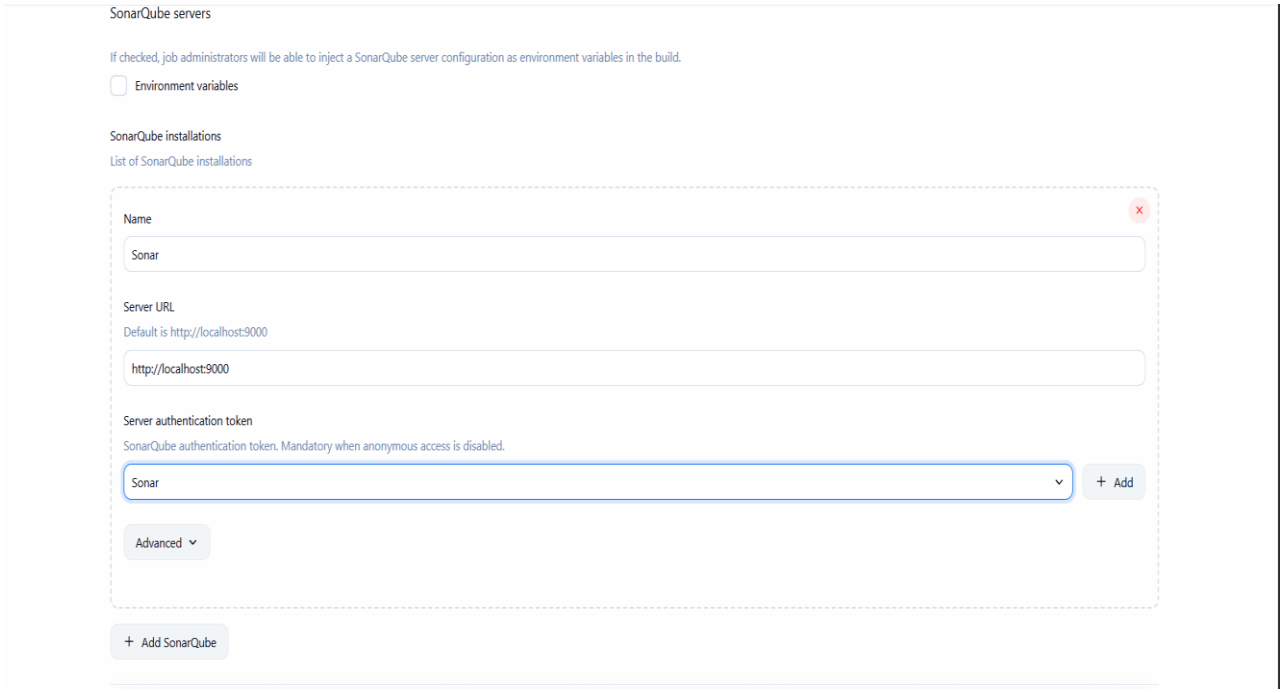


4)jenkins --> system --> SonarQube installations -->

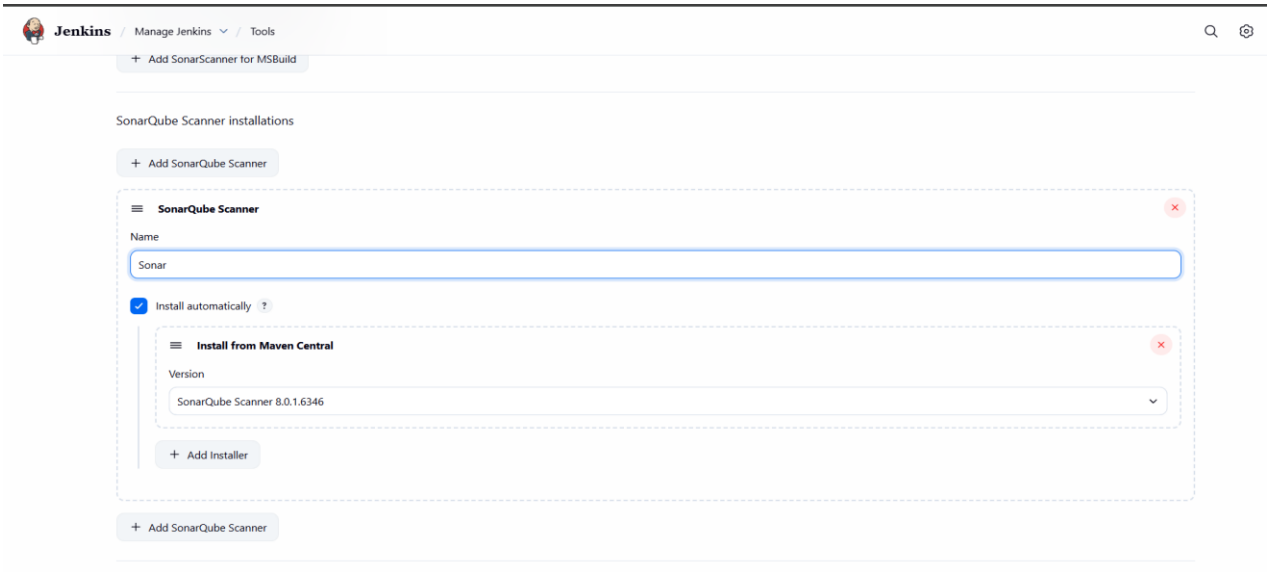
NAME: sonar ,

SERVER URL:http://35.154.237.213:9000 ,

SERVER_AUTH_Token :sonar



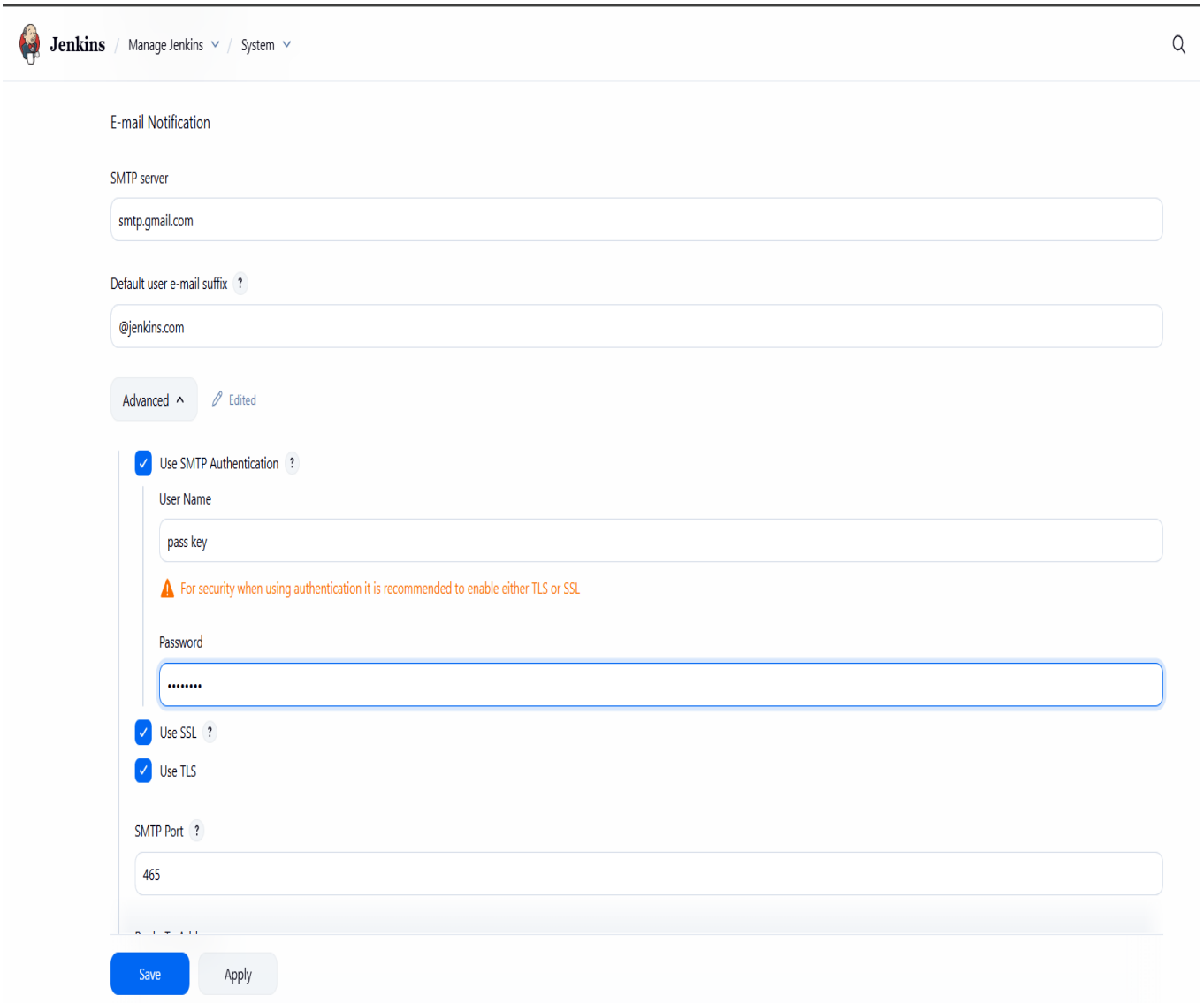
5)jenkins → tools →sonarqube scanner



6) jenkins --> tools --> dependency check

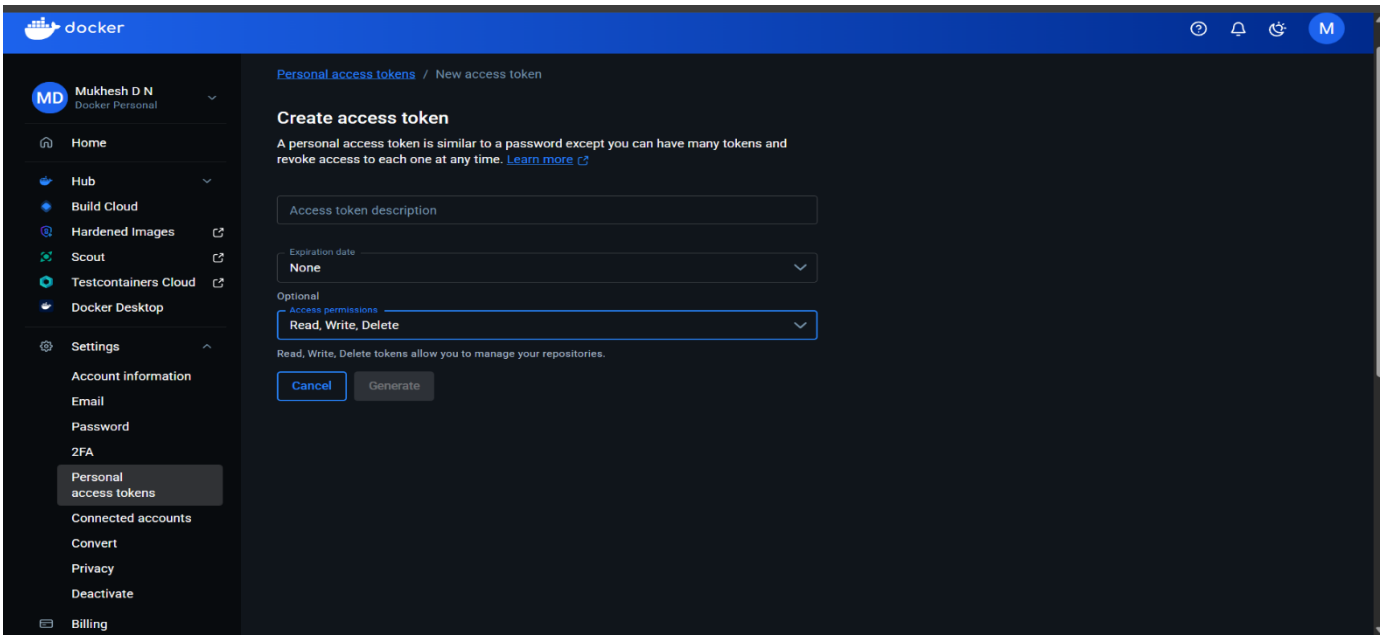
Email notification configuration

jenkins --> system --> email notification

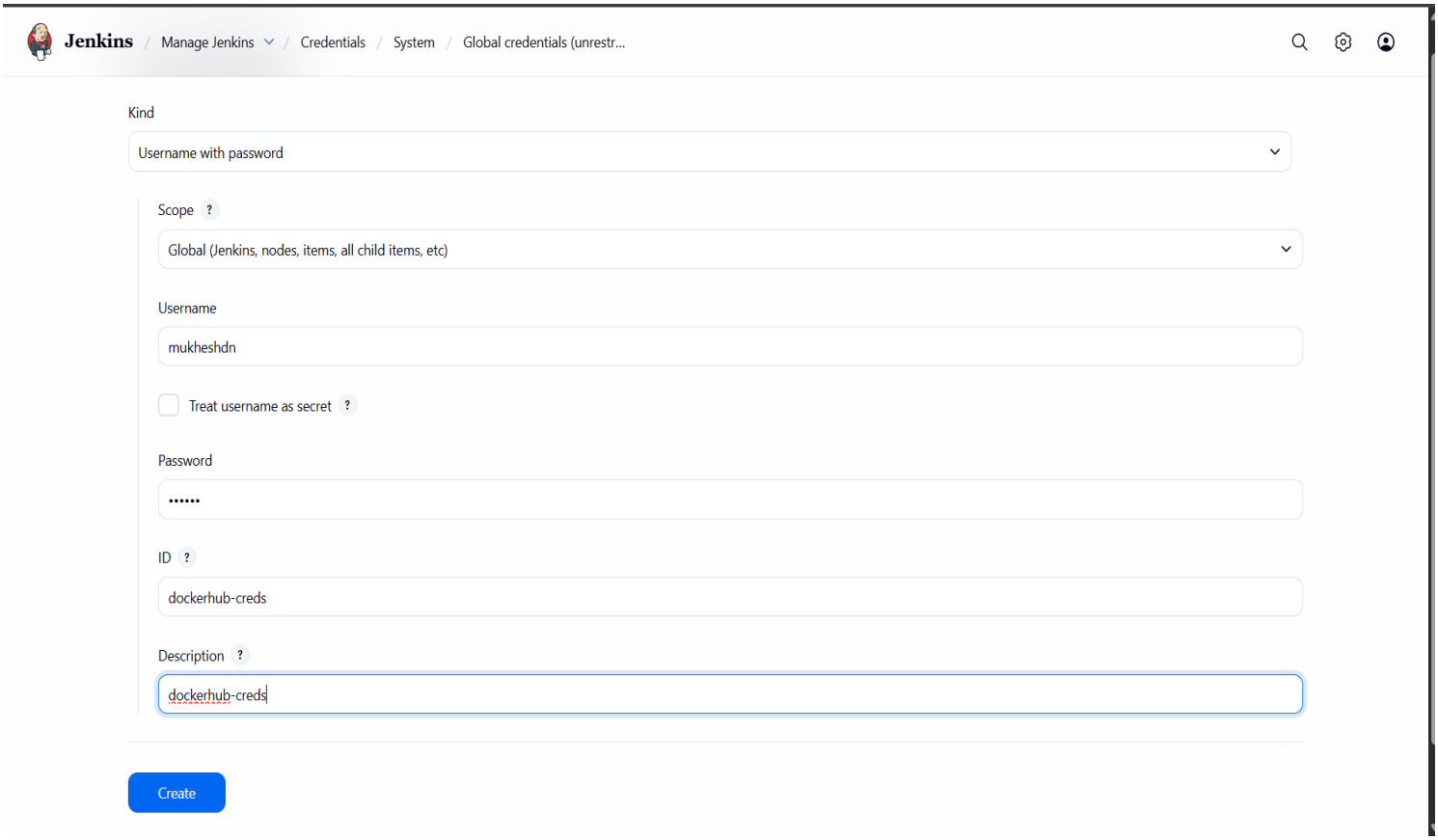


Docker and github credentials setup

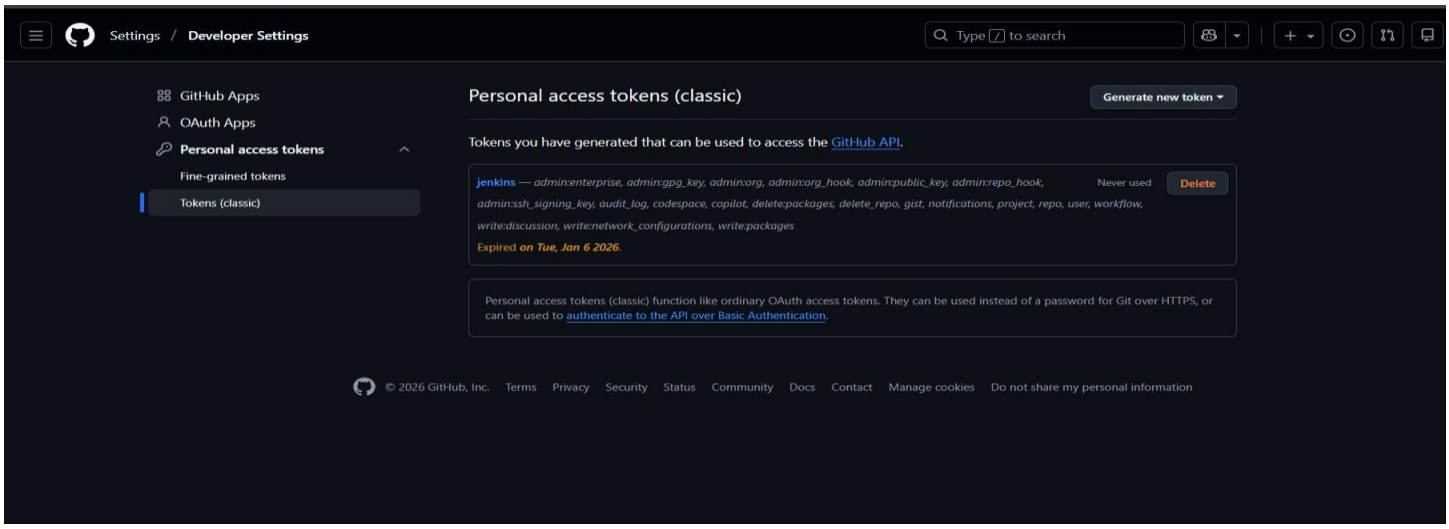
1) Create personal access token in docker hub

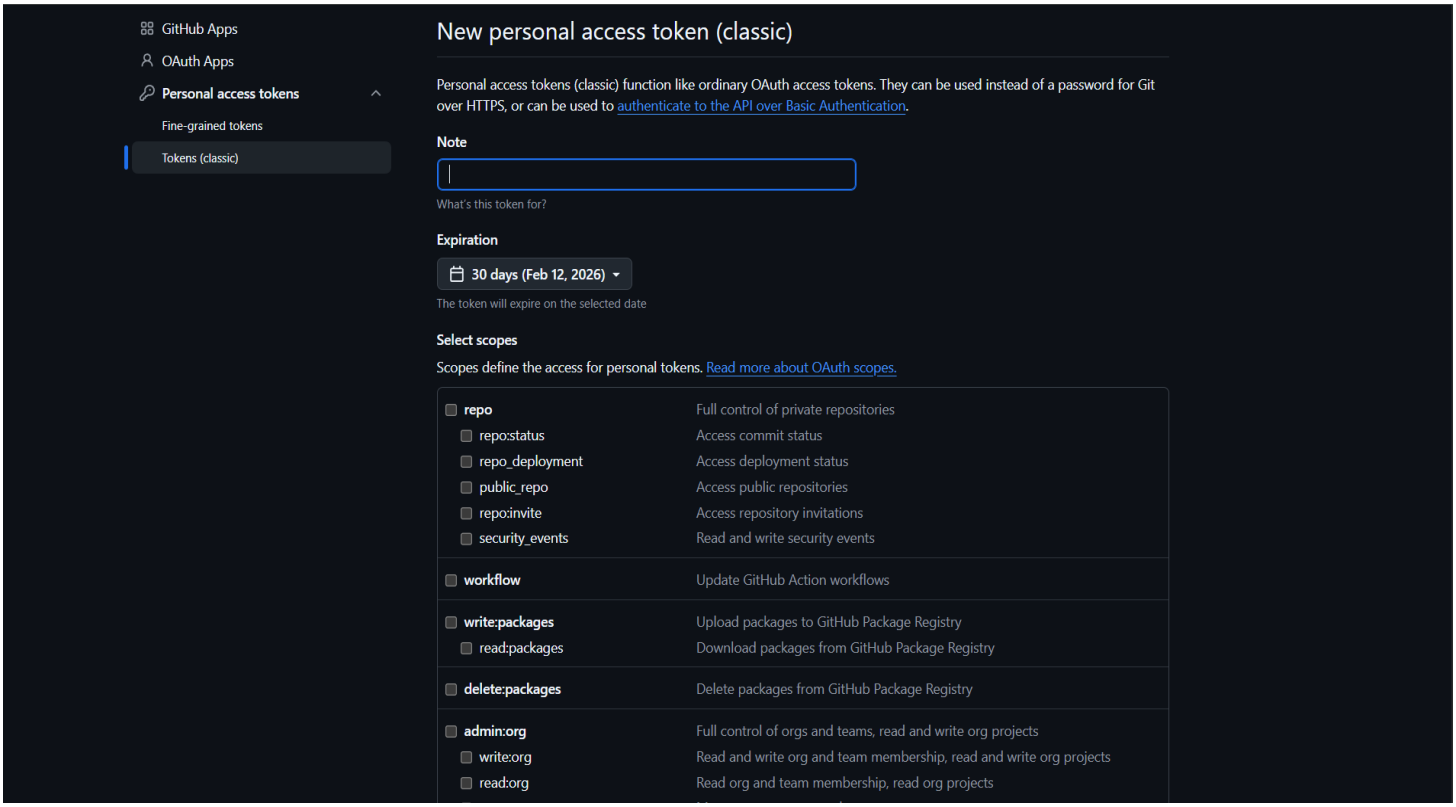


2) Add this in Jenkins credentials

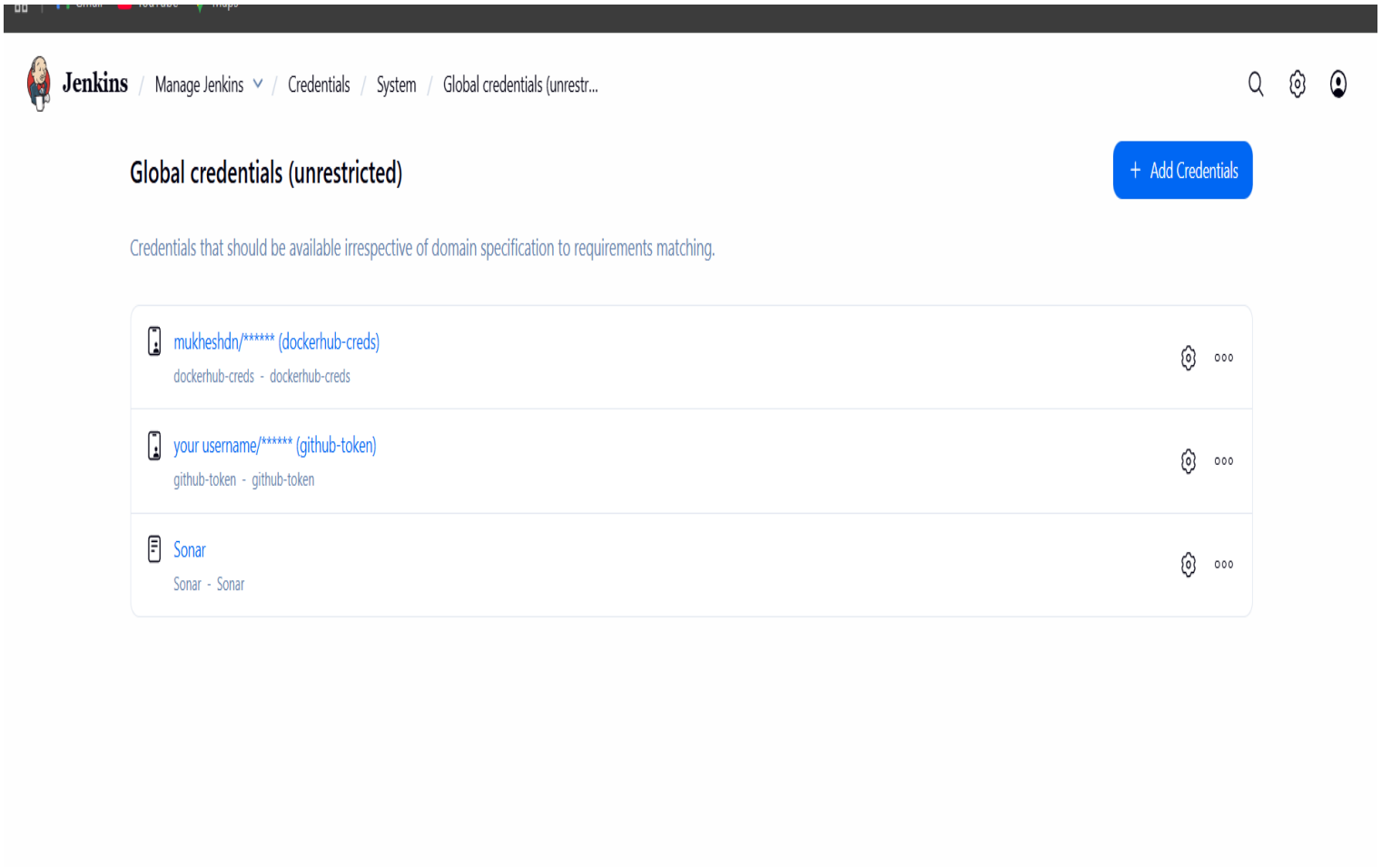


1) Create personal acces token (classic) in github





Final credentails in Jenkins

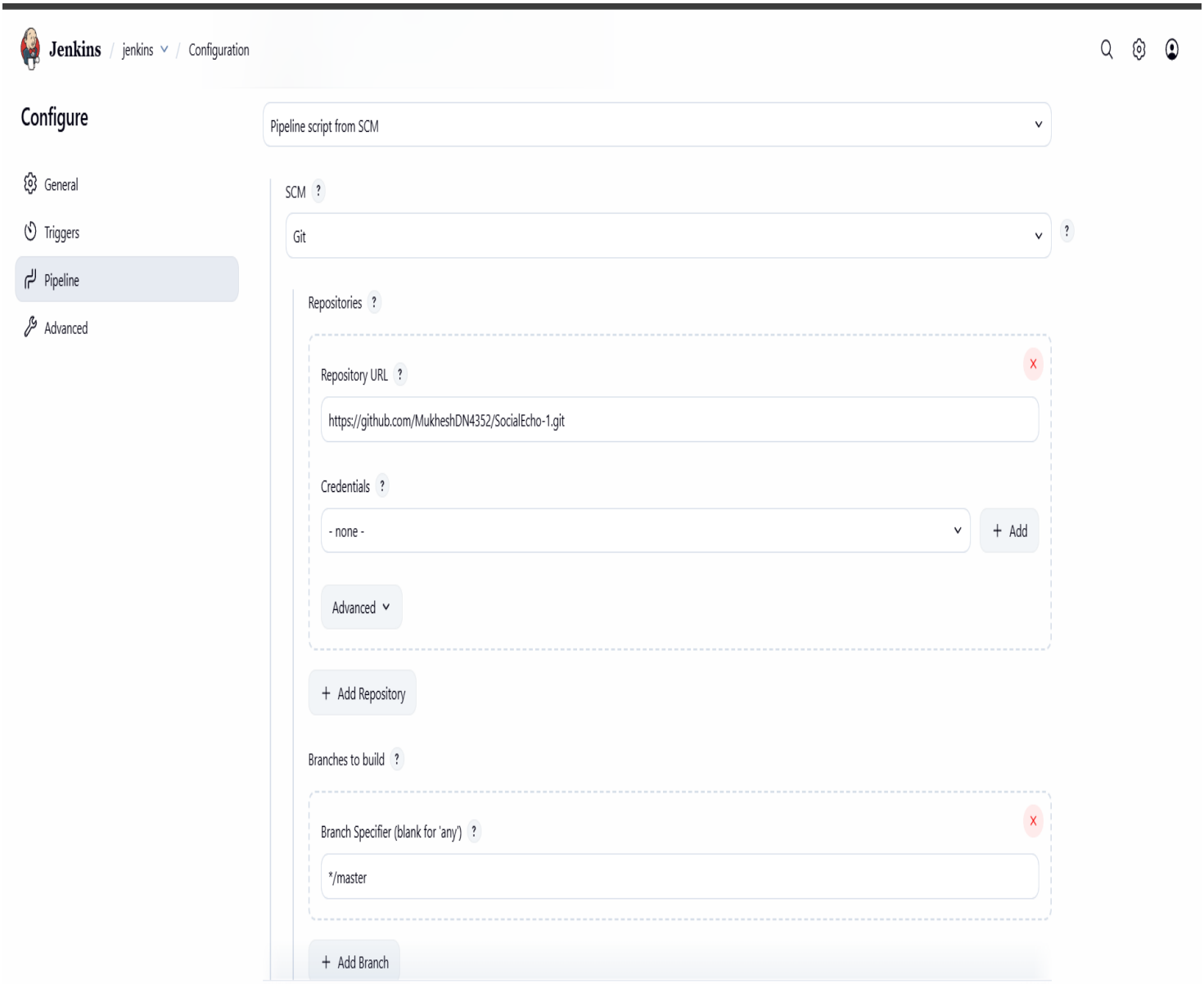


sudo usermod -aG docker jenkins

sudo systemctl restart jenkins

Jenkins pipeline setup

Select the job pipeline



jenkins file link

https://github.com/MukheshDN4352/DevSecOps-Project/blob/main/Jenkinsfile

Kubernetes setup

PreRequisites

Before setting up and configuring Kubernetes on AWS using Amazon EKS, ensure that the following prerequisites are in place:

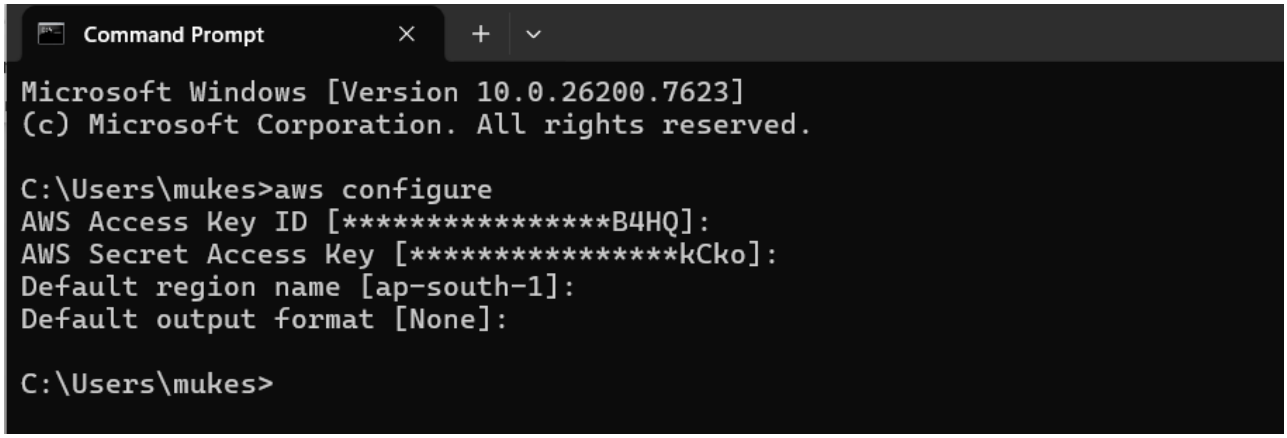
- **AWS Account**
  An active AWS account to provision and manage the EKS cluster and related cloud resources.
- **AWS CLI**
  The AWS Command Line Interface installed and configured with valid credentials and a default region.
- **EKS CLI (eksctl)**
  The eksctl tool installed to create and manage the Amazon EKS cluster.
- **kubectl**
  The Kubernetes command-line tool (kubectl) installed to interact with the EKS cluster.
- **Helm**
  The **Helm package manager** installed to deploy and manage Kubernetes applications and third-party tools (e.g., ArgoCD, Nginx Ingress, monitoring tools).

Kubernetes Setup Overview

- **Node Groups** were created for the EKS cluster to manage worker nodes and support scalable application workloads.
- **Persistent Volumes** were configured to provide durable storage for stateful components.
- The **AWS EBS CSI Driver** was installed and used to dynamically provision persistent volumes.
- **Ingress** was configured to manage external access to services running inside the cluster.
- An **AWS ALB Ingress Controller / AWS Load Balancer Controller** was deployed to provision and manage Application Load Balancers for Kubernetes services.
- **ArgoCD** was deployed on the cluster using **Helm** to enable GitOps-based continuous deployment.
- **Prometheus** and **Grafana** were deployed within the cluster for monitoring and observability.
- All required **commands and configuration steps** for the above components are provided in the sections below.

Cluster setup

1) Configure to your aws account



2) eksctl create cluster --name test --region ap-south-1 --without-nodegroup

Creating node groups

1. eksctl create nodegroup --cluster test --region ap-south-1 --name backend-ng --node-type c5.large --nodes 2 --nodes-min 2 --nodes-max 6 --node-volume-size 40 --node-volume-type gp3 --tags "project=socialecho,env=prod,component=backend" --node-labels "app=backend,tier=backend"

2. eksctl create nodegroup --cluster test --region ap-south-1 --name db-ng --node-type r5.large --nodes 1 --nodes-min 1 --nodes-max 1 --node-volume-size 100 --node-volume-type gp3 --tags "project=socialecho,env=prod,component=database" --node-labels "app=database,tier=storage"

3. eksctl create nodegroup --cluster test --region ap-south-1 --name backend-canary-ng --node-type c5.large --nodes 1 --nodes-min 1 --nodes-max 3 --node-volume-size 20 --node-volume-type gp3 --tags "project=socialecho,env=canary,component=backend-canary" --node-labels "app=backend,version=canary,tier=backend-canary"

4. eksctl create nodegroup --cluster test --region ap-south-1 --name frontend-canary-ng --node-type t3.medium --nodes 2 --nodes-min 1 --nodes-max 5 --node-volume-size 30 --node-volume-type gp3 --tags "project=socialecho,env=canary,component=frontend-canary" --node-labels "app=frontend,version=canary,tier=frontend-canary"

5. eksctl create nodegroup --cluster test --region ap-south-1 --name frontend-ng --node-type t3.medium --nodes 2 --nodes-min 2 --nodes-max 4 --node-volume-size 30 --node-volume-type gp3 --tags "project=socialecho,env=prod,component=frontend" --node-labels "app=frontend,tier=frontend"

associate oidc provider

eksctl utils associate-iam-oidc-provider --region=ap-south-1 --cluster=test –approve

installing CSI driver

eksctl create iamserviceaccount --name ebs-csi-controller-sa --namespace kube-system --cluster test --role-name AmazonEKS_EBS_CSI_DriverRole --role-only --attach-policy-arn arn:aws:iam::aws:policy/service-role/AmazonEBSCSIDriverPolicy –approve

eksctl create addon --name aws-ebs-csi-driver --cluster test --service-account-role-arn arn:aws:iam::**<AWS_ACCOUNT_ID>:**role/AmazonEKS_EBS_CSI_DriverRole –force

Apply all the deployments.yaml files in the order given below

kubectl apply -f namespace.yaml

kubectl apply -f secrets.yaml

kubectl apply -f mongo-server.yaml

kubectl apply -f mongo-express-server.yaml

kubectl apply -f node-canary.yaml

kubectl apply -f node-server.yaml

kubectl apply -f frontend-server.yaml

kubectl apply -f frontend-canary.yaml

Installing ALB controller

curl -O https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/v2.11.0/docs/install/iam_policy.json

aws iam create-policy --policy-name AWSLoadBalancerControllerIAMPolicy --policy-document file://iam_policy.json

eksctl create iamserviceaccount --cluster=test --namespace=kube-system --name=aws-load-balancer-controller --role-name AmazonEKSLoadBalancerControllerRole --attach-policy-arn=arn:aws:iam:: **<AWS_ACCOUNT_ID>::**policy/AWSLoadBalancerControllerIAMPolicy --approve

helm repo add eks https://aws.github.io/eks-charts

helm repo update eks

helm install aws-load-balancer-controller eks/aws-load-balancer-controller -n kube-system --set clusterName=test --set serviceAccount.create=false --set serviceAccount.name=aws-load-balancer-controller --set region=ap-south-1 --set vpcId=**<VPC_ID_OF_THE CLUSTER>**

verification

kubectl get deployment -n kube-system aws-load-balancer-controller

kubectl apply -f ingress.yaml

Access the application using the DNS name generated by the AWS Load Balancer.

ArgoCD setup

Prerequisites

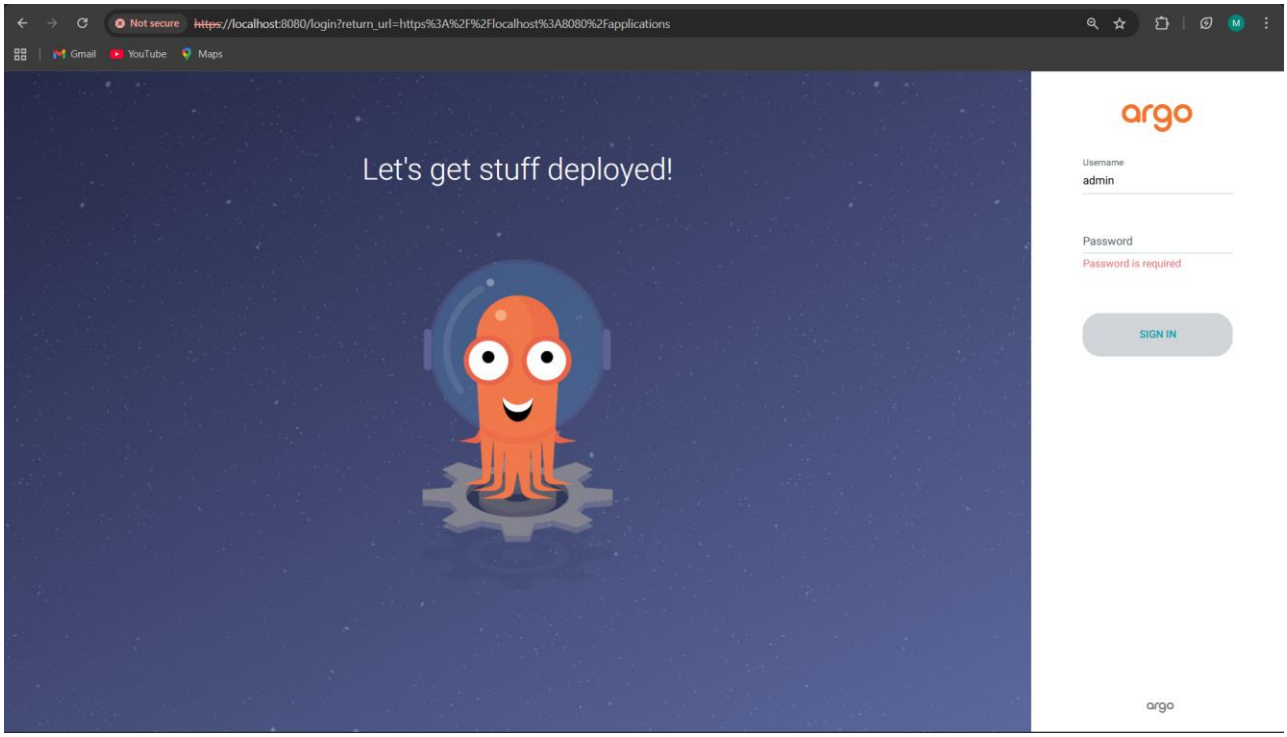
Before deploying and configuring ArgoCD in the Kubernetes cluster, ensure that the following prerequisites are in place:

- Active Kubernetes Cluster
  A running and accessible Kubernetes cluster (Amazon EKS) with worker nodes in a Ready state.
- kubectl Configured
  The kubectl CLI installed and configured with the correct kubeconfig to access the target cluster.

Commands

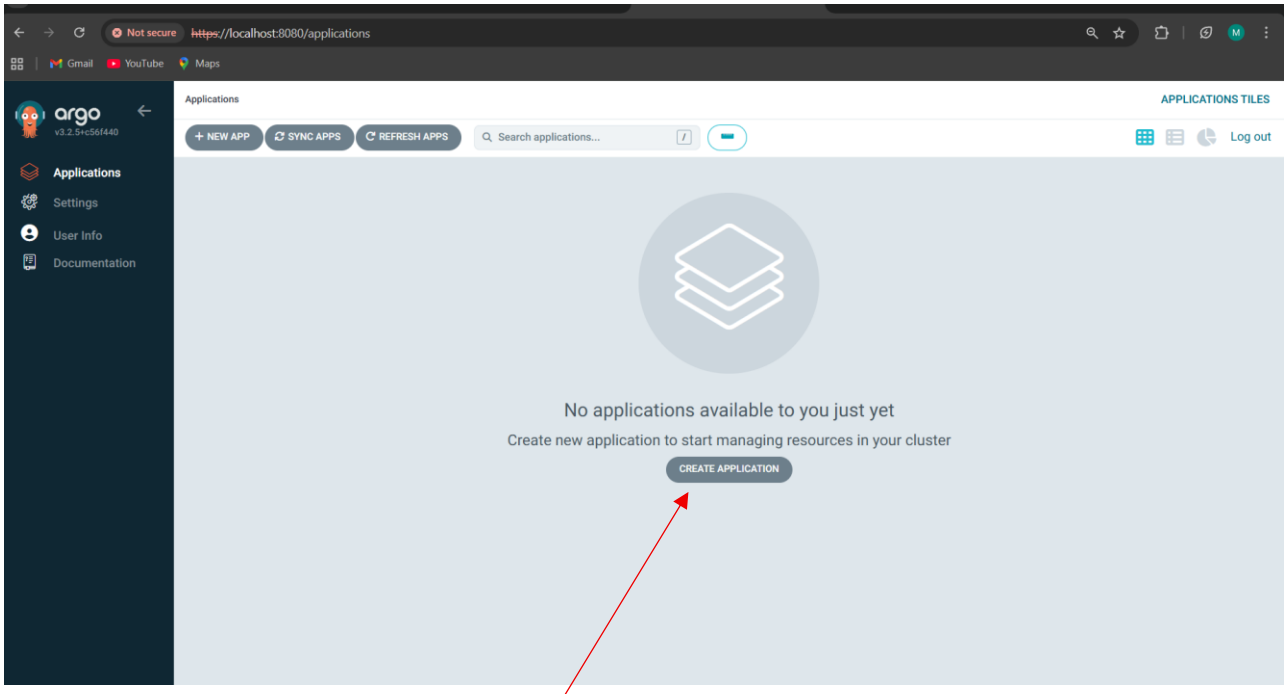
- **Kubectl create namespace argocd**
- **kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml**
- **kubectl get pods -n argocd**
- **kubectl port-forward svc/argocd-server -n argocd 8080:443**

Access the application in browser via http://localhost:8080

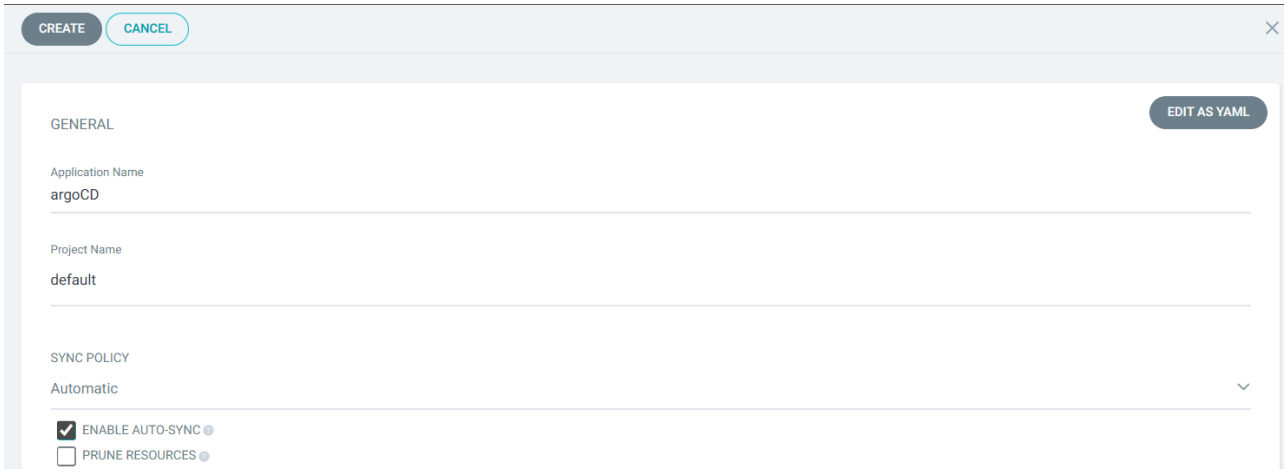


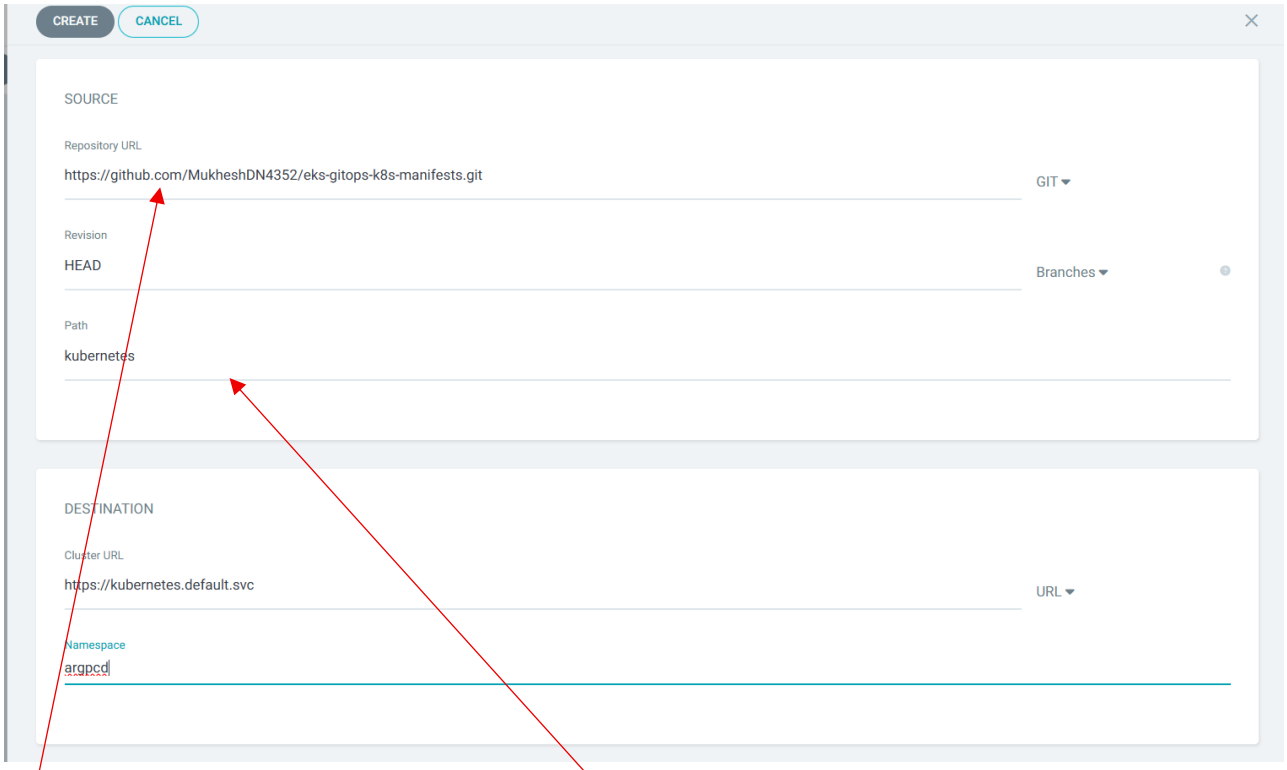
To get the password

kubectl get secret argocd-initial-admin-secret -n argocd -o jsonpath="{.data.password}" | base64 –decode



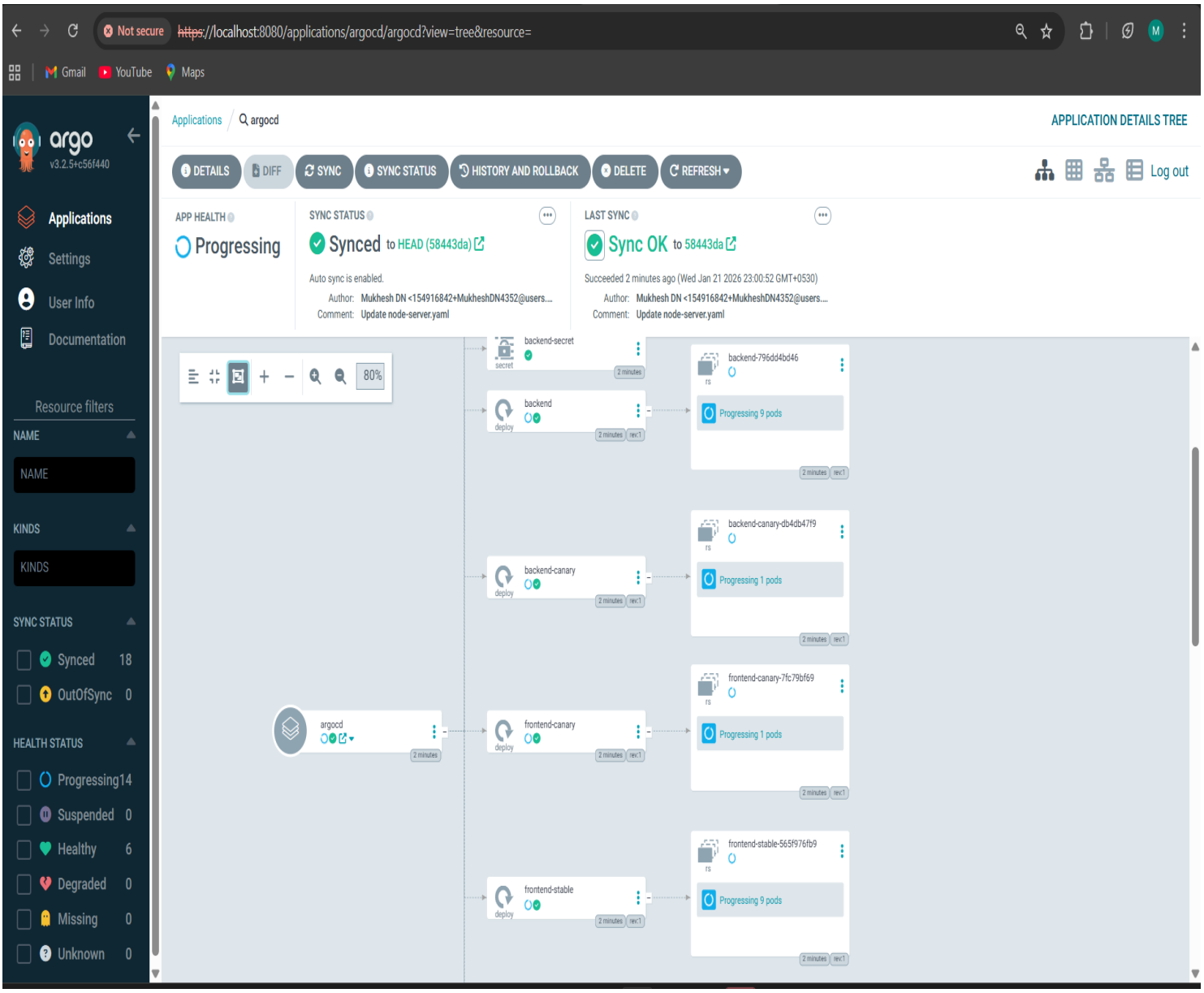
Click **create application**





Give you kubernetes git hub repo manifest and give the correct path where your yaml files are present

Click on **Create**


Not secure
https://localhost:8080/applications/argocd/argocd?view=tree&resource=
Gmail
YouTube
Maps
argo
v3.2.5+c56f440
Applications
Settings
User Info
Documentation
Resource filters
NAME
KINDS
SYNC STATUS
Synced 18
OutOfSync 0
HEALTH STATUS
Progressing 14
Suspended 0
Healthy 6
Degraded 0
Missing 0
Unknown 0
Applications
argocd
APPLICATION DETAILS TREE
DETAILS
DIFF
SYNC
SYNC STATUS
HISTORY AND ROLLBACK
DELETE
REFRESH
Log out
APP HEALTH
Progressing
SYNC STATUS
Synced to HEAD (58443da)
Auto sync is enabled.
Author: Mukhesh DN <154916842+MukheshDN4352@users....
Comment: Update node-server.yaml
LAST SYNC
Sync OK to 58443da
Succeeded 2 minutes ago (Wed Jan 21 2026 23:00:52 GMT+0530)
Author: Mukhesh DN <154916842+MukheshDN4352@users....
Comment: Update node-server.yaml
80%
backend-secret
backend
backend-796dd4bd46
Progressing 9 pods
backend-canary
backend-canary-db4db47f9
Progressing 1 pods
argocd
frontend-canary
frontend-canary-7fc79bf69
Progressing 1 pods
frontend-stable
frontend-stable-565f976fb9
Progressing 9 pods

Prometheus and Grafana

Prerequisites

Before deploying and configuring Prometheus and Grafana in the Kubernetes cluster, ensure that the following prerequisites are in place:

- **Active Kubernetes Cluster (EKS)**
  A running and accessible Amazon EKS cluster with all worker nodes in a Ready state.
- **kubectl Configured**
  The kubectl CLI installed and configured with the correct kubeconfig to communicate with the EKS cluster.
- **Helm Installed**
  The Helm package manager installed to deploy Prometheus and Grafana using Helm charts.

Commands

1. helm repo add stable https://charts.helm.sh/stable
2. helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
3. kubectl create namespace prometheus
4. helm install stable prometheus-community/kube-prometheus-stack -n prometheus
5. kubectl get pods -n prometheus

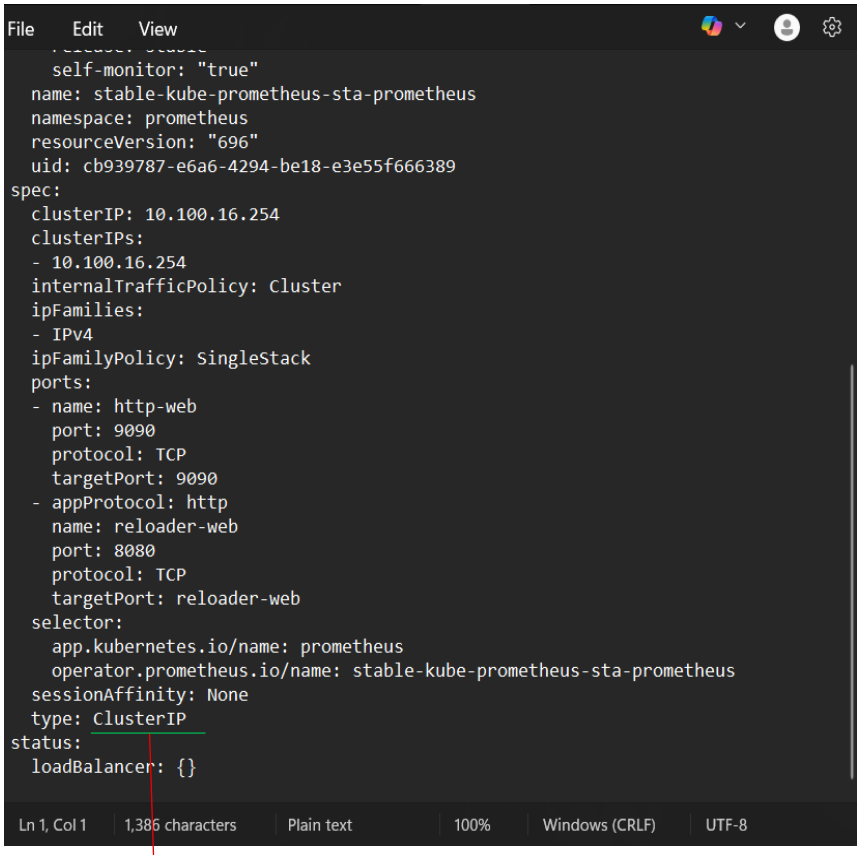
```
C:\Users\mukes>kubectl get pods -n prometheus
NAME                                                     READY   STATUS    RESTARTS   AGE
alertmanager-stable-kube-prometheus-sta-alertmanager-0   2/2     Running   0          106s
prometheus-stable-kube-prometheus-sta-prometheus-0       2/2     Running   0          105s
stable-grafana-646cfcfc5f-q597k                          3/3     Running   0          2m8s
stable-kube-prometheus-sta-operator-667c9d44d4-4fnjb     1/1     Running   0          2m8s
stable-kube-state-metrics-599d5b459c-nvtx5               1/1     Running   0          2m8s
stable-prometheus-node-exporter-lqq4z                    1/1     Running   0          2m8s
```

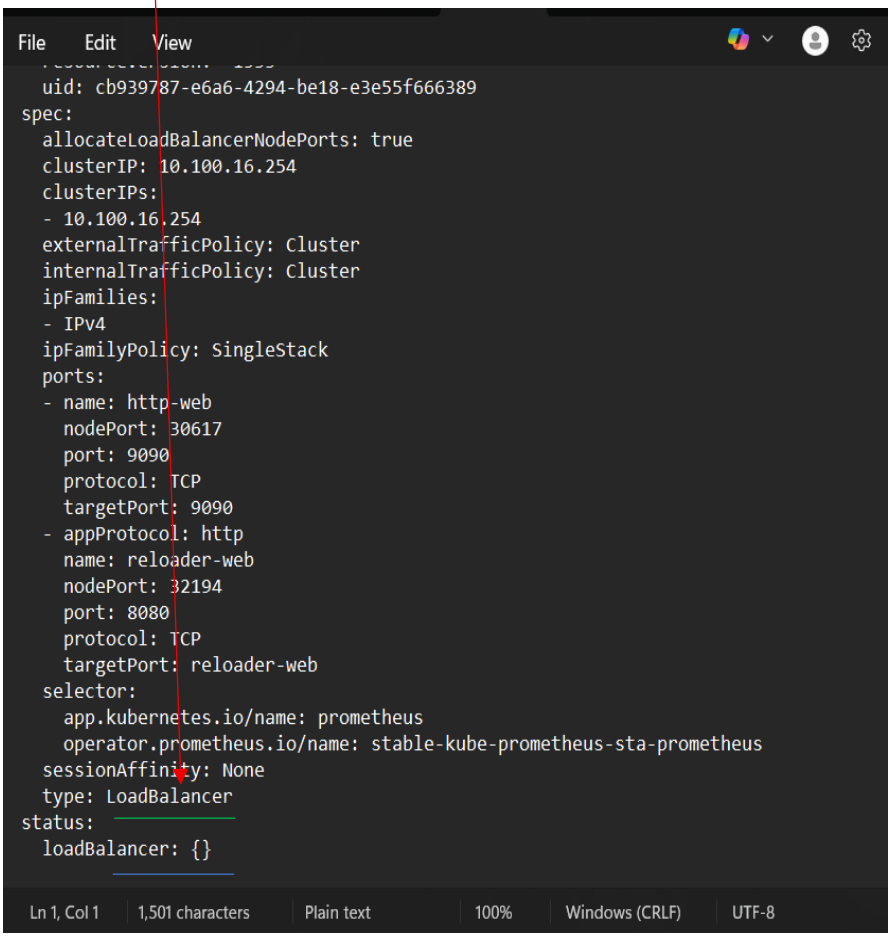
6.kubectl get svc -n prometheus

```
C:\Users\mukes>kubectl get svc -n prometheus
NAME                                      TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)                      AGE
alertmanager-operated                     ClusterIP   None             <none>        9093/TCP,9094/TCP,9094/UDP   2m47s
prometheus-operated                       ClusterIP   None             <none>        9090/TCP                     2m46s
stable-grafana                            ClusterIP   10.105.113.154   <none>        80/TCP                       3m9s
stable-kube-prometheus-sta-alertmanager   ClusterIP   10.109.153.172   <none>        9093/TCP,8080/TCP            3m9s
stable-kube-prometheus-sta-operator       ClusterIP   10.101.133.143   <none>        443/TCP                      3m9s
stable-kube-prometheus-sta-prometheus     ClusterIP   10.100.16.254    <none>        9090/TCP,8080/TCP            3m9s
stable-kube-state-metrics                 ClusterIP   10.102.8.61      <none>        8080/TCP                     3m9s
stable-prometheus-node-exporter           ClusterIP   10.102.30.185    <none>        9100/TCP                     3m9s
```

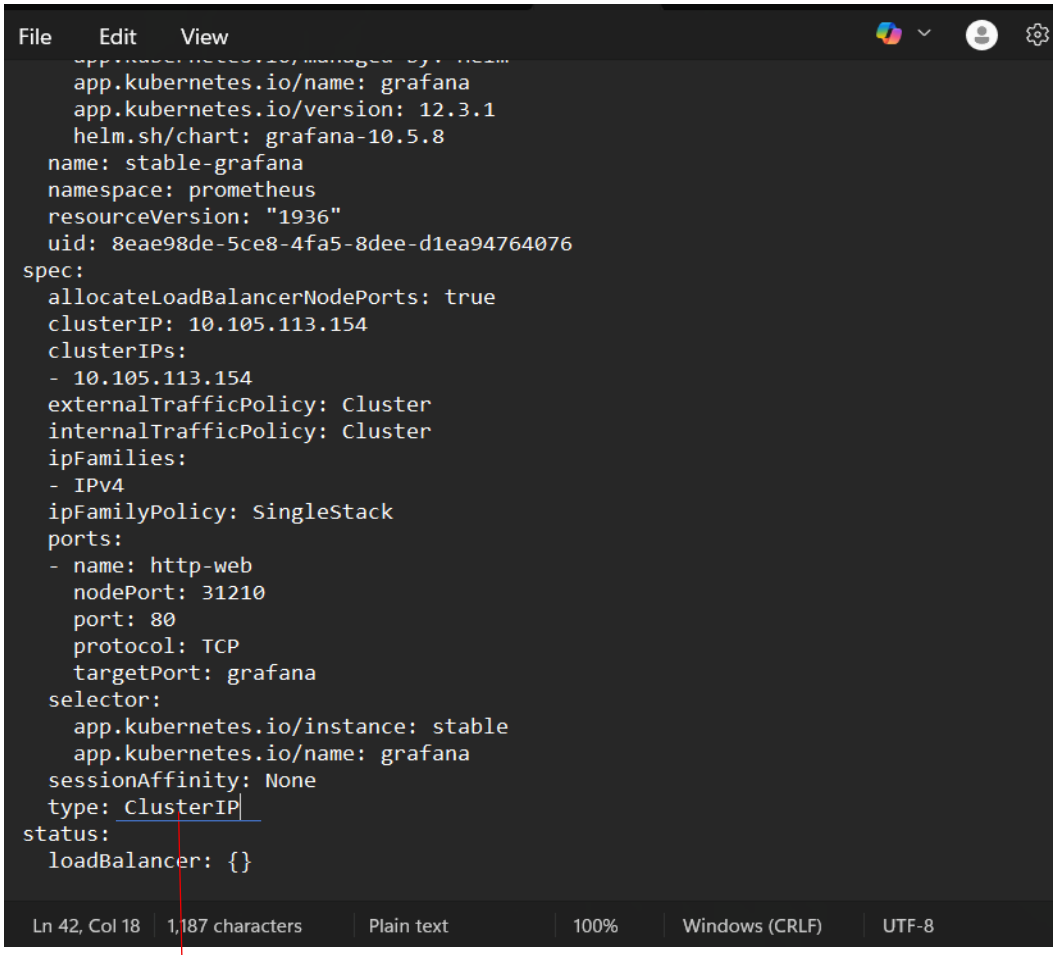
Expose prometheus and grafana to access

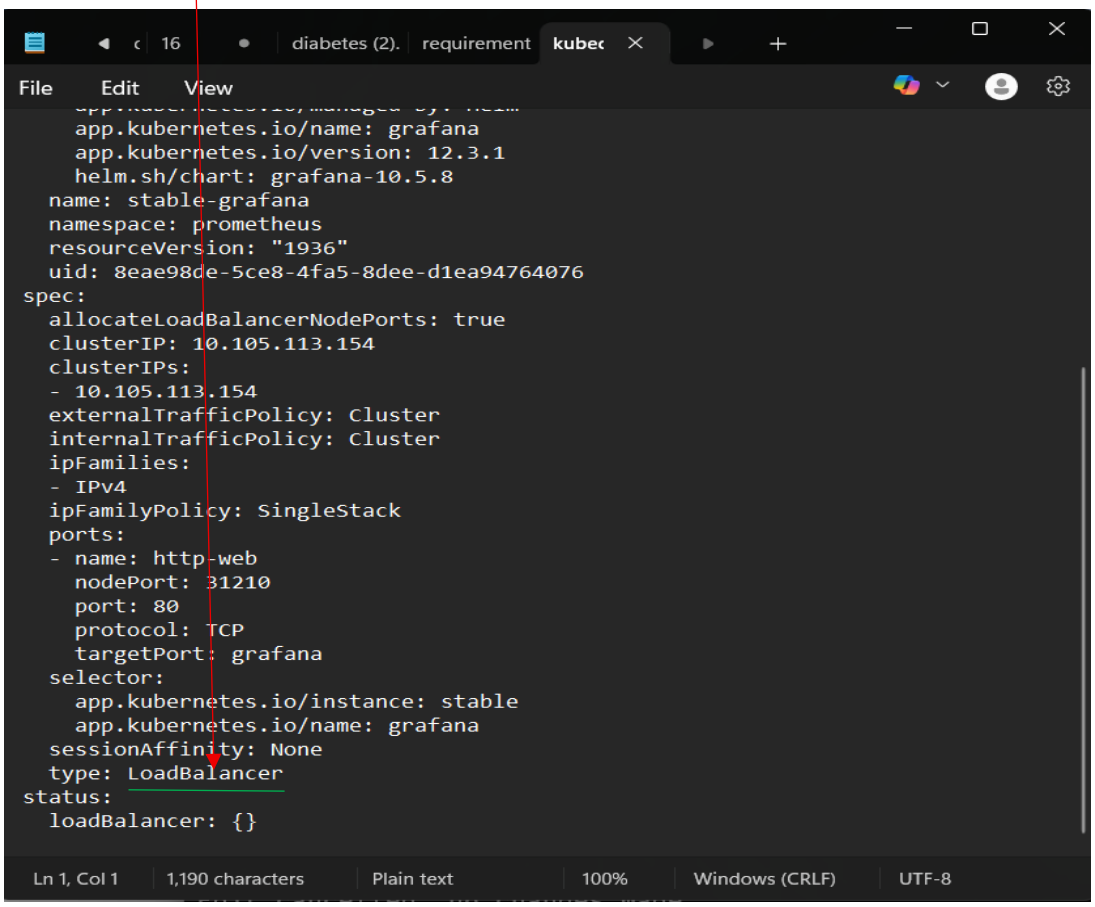
kubectl edit svc stable-kube-prometheus-sta-prometheus -n prometheus





kubectl edit svc stable-grafana -n prometheus





Access the both the dashboards with **external Ip** created by **LoadBalancer**

The complete cluster data can be visualized in Grafana dashboards, including all metrics related to pod health and performance.

Thank you